

TRƯỜNG ĐẠI HỌC KHOA HỌC - ĐẠI HỌC HUẾ

KHOA CÔNG NGHỆ THÔNG TIN



**ĐỀ TÀI: HỆ THỐNG KHOÁ CỬA THÔNG MINH
TRÊN ESP32 VÀ RFID**

Học phần : Phát triển ứng dụng IoT

Giáo viên hướng dẫn : Võ Việt Dũng

Họ và tên : Nguyễn Gia Huy

Mã số sinh viên : 22T1020151

Lớp : CNTT - K46H

Nhóm : 02

Chuyên ngành : Công nghệ phần mềm

Năm học 2025 – 2026

Mục Lục

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

- 1.1. Tổng quan về hệ thống khóa cửa thông minh
 - 1.1.1. Khái niệm hệ thống khóa cửa thông minh
 - 1.1.2. Ưu điểm của hệ thống
 - 1.1.3. Ứng dụng thực tế
- 1.2. Vi điều khiển ESP32
 - 1.2.1. Giới thiệu chung
 - 1.2.2. Một số đặc điểm nổi bật
 - 1.2.3. Vai trò trong hệ thống
- 1.3. Công nghệ RFID
 - 1.3.1. Khái niệm
 - 1.3.2. Cấu trúc hệ thống RFID
 - 1.3.3. Nguyên lý hoạt động
 - 1.3.4. Ưu điểm của RFID
 - 1.3.5. Vai trò trong đề tài
- 1.4. Module RFID RC522
 - 1.4.1. Giới thiệu module
 - 1.4.2. Giao tiếp với ESP32
 - 1.4.3. Các chân kết nối
 - 1.4.4. Vai trò trong hệ thống
- 1.5. Servo motor
 - 1.5.1. Giới thiệu
 - 1.5.2. Nguyên lý hoạt động
 - 1.5.3. Vai trò trong đề tài

1.6. Wi-Fi trong hệ thống

1.6.1. Giới thiệu

1.6.2. Vai trò trong hệ thống

1.6.3. Quy trình xác thực

1.7. Telegram Bot

1.7.1. Giới thiệu

1.7.2. Chức năng trong hệ thống

1.7.3. Vai trò trong đề tài

1.8. LED và buzzer

1.8.1. Giới thiệu

1.8.2. Vai trò trong hệ thống

1.9. Kết luận chương

CHƯƠNG 2: PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu bài toán

2.1.1. Mục tiêu hệ thống

2.1.2. Các yêu cầu cơ bản

2.2. Yêu cầu chức năng

2.2.1. Chức năng nhận diện thẻ RFID

2.2.2. Chức năng xác thực hai lớp

2.2.3. Chức năng mở và đóng cửa

2.2.4. Chức năng hiển thị trạng thái

2.2.5. Chức năng giám sát và điều khiển từ xa

2.3. Yêu cầu phi chức năng

2.4. Sơ đồ khối hệ thống

2.5. Nguyên lý hoạt động của hệ thống

- 2.6. Thiết kế sơ đồ kết nối phần cứng
 - 2.6.1. Kết nối RFID RC522 với ESP32
 - 2.6.2. Kết nối servo
 - 2.6.3. Kết nối LED và buzzer

2.7. Thiết kế phần mềm

2.8. Lưu đồ thuật toán

2.9. Kết luận chương

CHƯƠNG 3: THI CÔNG HỆ THỐNG

3.1. Giới thiệu chương

3.2. Danh sách linh kiện sử dụng

3.3. Thi công phần cứng

- 3.3.1. Vai trò các khối phần cứng

- 3.3.2. Sơ đồ kết nối phần cứng

- 3.3.3. Mô tả hoạt động phần cứng

3.4. Thi công phần mềm

- 3.4.1. Môi trường phát triển

- 3.4.2. Cấu trúc chương trình

- 3.4.3. Các hàm chính

- 3.4.4. Nguyên lý hoạt động phần mềm

3.5. Cơ chế xác thực và điều khiển qua Wi-Fi bằng Telegram

3.6. Mô phỏng hệ thống trên Wokwi

3.7. Khó khăn trong quá trình thực hiện

3.8. Kết luận chương

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Giới thiệu chương

4.2. Môi trường thực nghiệm

4.3. Các trường hợp kiểm thử

4.4. Bảng tổng hợp kết quả kiểm thử

4.5. Đánh giá hệ thống

4.5.1. Ưu điểm

4.5.2. Hạn chế

4.5.3. Mức độ đáp ứng yêu cầu đề tài

4.6. Kết luận chương

CHƯƠNG 5: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

5.1. Kết luận

5.2. Hướng phát triển

TÀI LIỆU THAM KHẢO

PHỤ LỤC

LỜI MỞ ĐẦU

Trong thời đại Cách mạng Công nghiệp 4.0, Internet vạn vật (IoT) đang ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống. Một trong những ứng dụng phổ biến là xây dựng các hệ

thông nhà thông minh nhằm nâng cao tiện nghi và đảm bảo an toàn cho người sử dụng.

Trong đó, hệ thống khóa cửa thông minh đóng vai trò quan trọng trong việc kiểm soát truy cập. Các phương pháp truyền thống như khóa cơ tồn tại nhiều hạn chế như dễ mất chìa khóa, khó quản lý và thiếu tính bảo mật. Do đó, việc ứng dụng công nghệ RFID kết hợp với kết nối không dây để xây dựng hệ thống khóa cửa thông minh là một hướng đi phù hợp.

Đề tài “Hệ thống khóa cửa thông minh dựa trên ESP32 và RFID, tích hợp xác thực qua Wi-Fi” được thực hiện nhằm xây dựng một mô hình khóa cửa có khả năng nhận diện thẻ từ và xác thực bổ sung thông qua Telegram. Qua đó, hệ thống giúp tăng cường bảo mật, cho phép giám sát và điều khiển từ xa một cách hiệu quả.

1. Mục tiêu đề tài.

Hệ thống được xây dựng nhằm mục tiêu thiết kế và mô phỏng một mô hình khóa cửa thông minh sử dụng ESP32 làm trung tâm điều khiển, kết hợp công nghệ RFID để nhận diện người dùng và tích hợp xác thực bổ sung qua Wi-Fi. Bên cạnh đó, đề tài hướng đến việc xây dựng một giải pháp có khả năng giám sát, cảnh báo và điều khiển từ xa thông qua Telegram, qua đó nâng cao tính tiện lợi và mức độ bảo mật cho hệ thống khóa cửa.

2. Phạm vi nghiên cứu.

Đề tài tập trung nghiên cứu và triển khai mô hình khóa cửa thông minh trên nền tảng ESP32, sử dụng module RFID RC522 để nhận diện thẻ, servo để mô phỏng cơ cấu đóng mở cửa, LED và buzzer để hiển thị và cảnh báo trạng thái. Hệ thống được tích hợp giao diện web xác thực mã PIN qua Wi-Fi và chức năng thông báo, điều khiển từ xa qua Telegram. Phần thực nghiệm

được tiến hành chủ yếu trên môi trường mô phỏng Wokwi kết hợp với phần mềm lập trình Arduino/PlatformIO.

3. Phương pháp nghiên cứu.

Đề tài được thực hiện thông qua việc tìm hiểu tài liệu về ESP32, RFID, giao tiếp Wi-Fi và hệ thống khóa thông minh; phân tích yêu cầu bài toán; thiết kế phần cứng và phần mềm; xây dựng mô hình mô phỏng trên Wokwi; lập trình điều khiển hệ thống; sau đó kiểm thử và đánh giá kết quả hoạt động trong các tình huống khác nhau.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về hệ thống khóa cửa thông minh

1.1.1. Khái niệm hệ thống khóa cửa thông minh

Hệ thống khóa cửa thông minh là giải pháp ứng dụng công nghệ điện tử, vi điều khiển và truyền thông không dây nhằm thay thế hoặc hỗ trợ cho các loại khóa truyền thống. Hệ thống cho phép sử dụng nhiều phương thức xác thực như thẻ từ RFID, mã PIN hoặc điều khiển từ xa thông qua Internet.

1.1.2. Ưu điểm của hệ thống

So với khóa cơ truyền thống, hệ thống khóa thông minh có nhiều ưu điểm như:

- Tăng tính tiện lợi trong sử dụng
- Hỗ trợ quản lý truy cập
- Nâng cao mức độ an toàn
- Có khả năng tích hợp với các hệ thống IoT

1.1.3. Ứng dụng thực tế

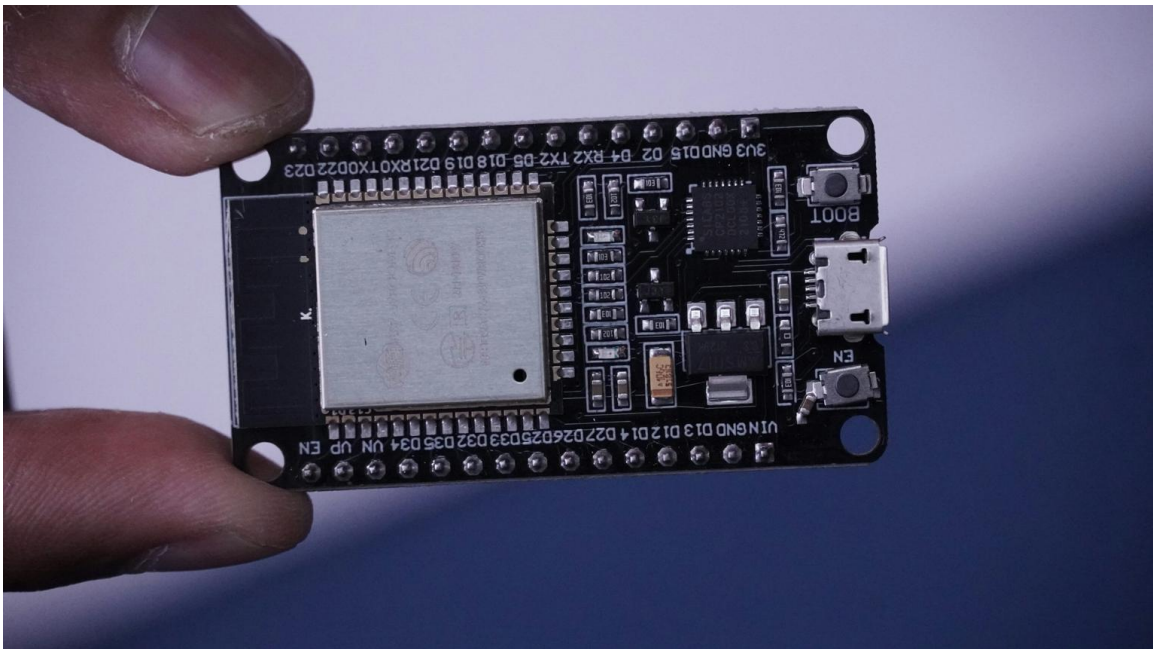
Hệ thống khóa cửa thông minh được ứng dụng rộng rãi trong:

- Nhà ở
- Văn phòng
- Phòng thí nghiệm
- Khu vực hạn chế truy cập

1.2. Vi điều khiển ESP32

1.2.1. Giới thiệu chung

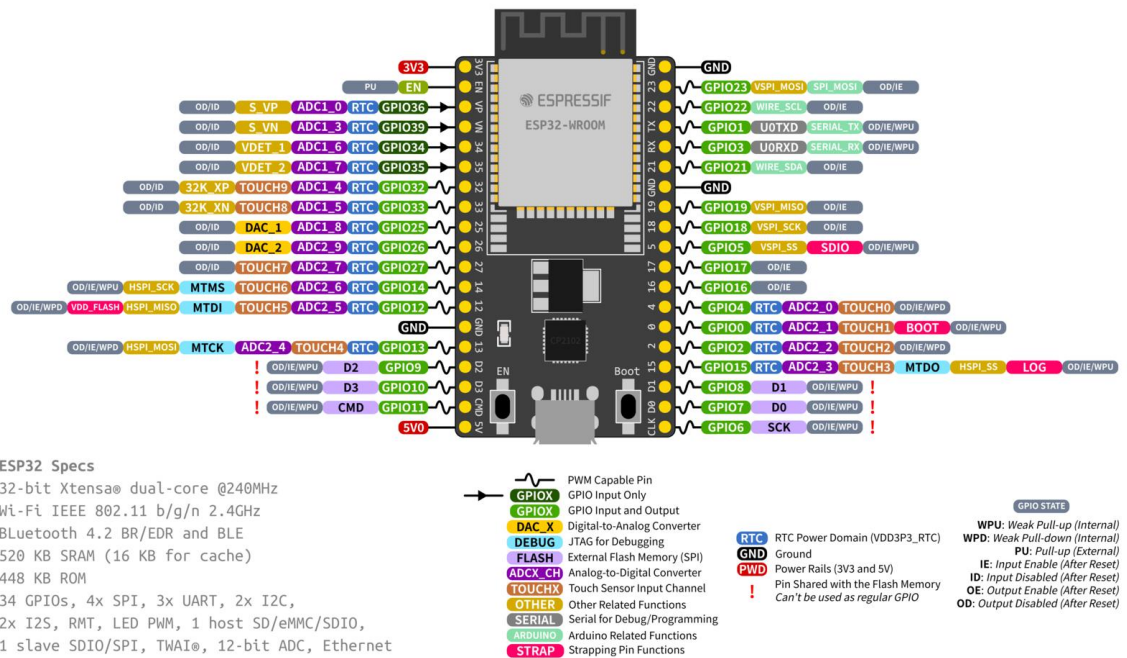
ESP32 là vi điều khiển tích hợp Wi-Fi và Bluetooth do Espressif Systems phát triển, được sử dụng phổ biến trong các ứng dụng IoT.



1.2.2. Một số đặc điểm nổi bật

ESP32 có các đặc điểm nổi bật sau:

- Tích hợp Wi-Fi và Bluetooth
- Nhiều chân GPIO
- Hỗ trợ SPI, UART, I2C, PWM
- Hiệu năng xử lý tốt
- Dễ lập trình với Arduino/PlatformIO



1.2.3. Vai trò trong hệ thống

Trong đề tài, ESP32 đóng vai trò trung tâm, thực hiện:

- Đọc dữ liệu từ RFID
- Xử lý xác thực
- Điều khiển servo
- Giao tiếp với Telegram

1.3. Công nghệ RFID

1.3.1. Khái niệm

RFID là công nghệ nhận dạng bằng sóng vô tuyến, cho phép nhận diện đối tượng mà không cần tiếp xúc.

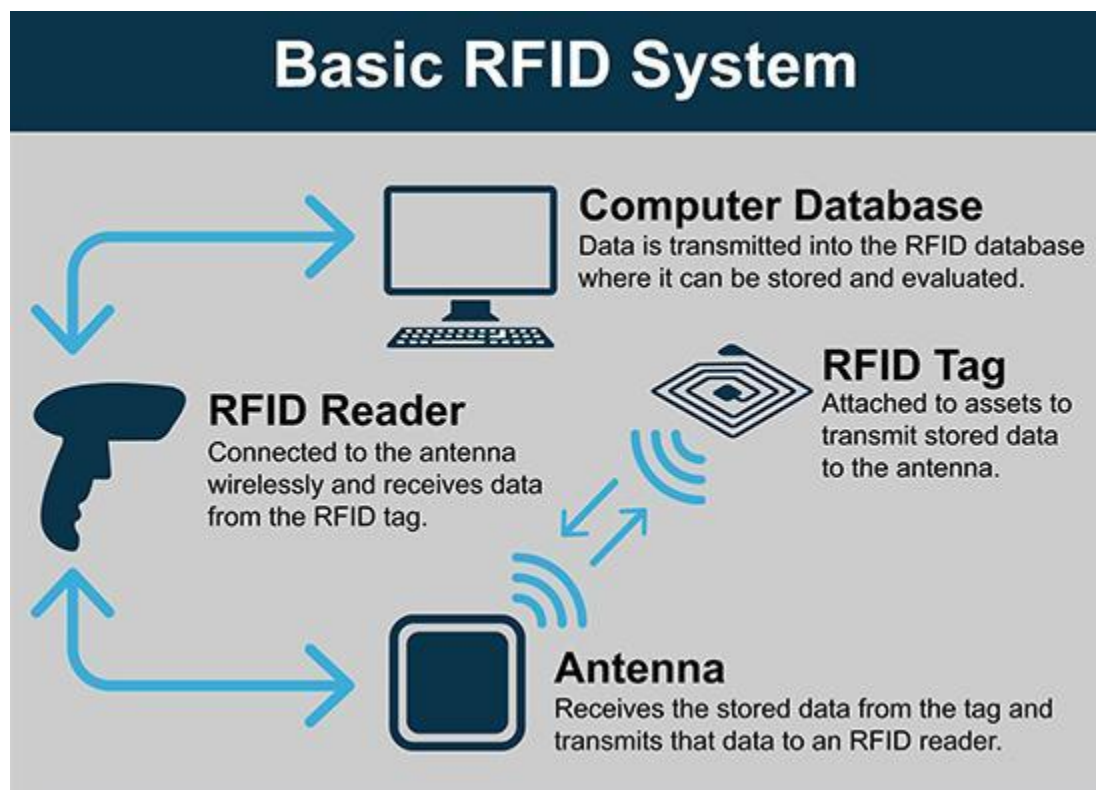
1.3.2. Cấu trúc hệ thống RFID

Hệ thống RFID gồm:

- Thẻ RFID
- Đầu đọc RFID

1.3.3. Nguyên lý hoạt động

Khi thẻ đi vào vùng quét, đầu đọc nhận UID và gửi về vi điều khiển để xử lý.



1.3.4. Ưu điểm của RFID

- Nhận diện nhanh
- Không cần tiếp xúc

- Chi phí thấp
- Dễ triển khai

1.3.5. Vai trò trong đề tài

RFID được sử dụng làm lớp xác thực đầu tiên

1.4. Module RFID RC522

1.4.1. Giới thiệu module

RC522 là module RFID hoạt động ở tần số 13.56 MHz.



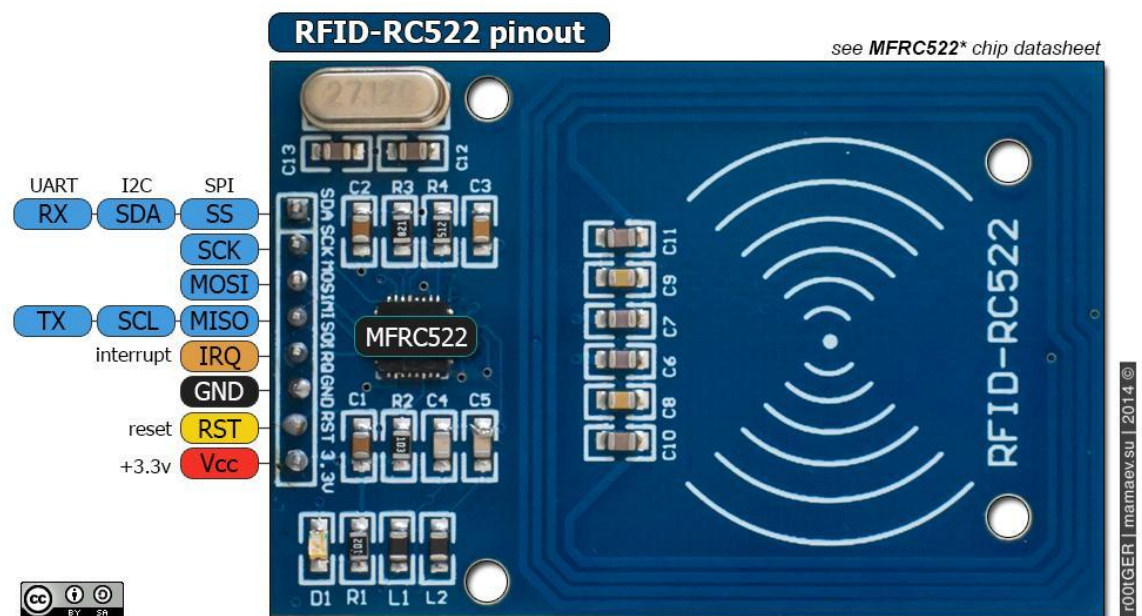
1.4.2. Giao tiếp với ESP32

RC522 sử dụng giao tiếp SPI để truyền dữ liệu.

1.4.3. Các chân kết nối

Các chân chính gồm:

- SDA
- SCK
- MOSI
- MISO
- RST
- 3.3V
- GND



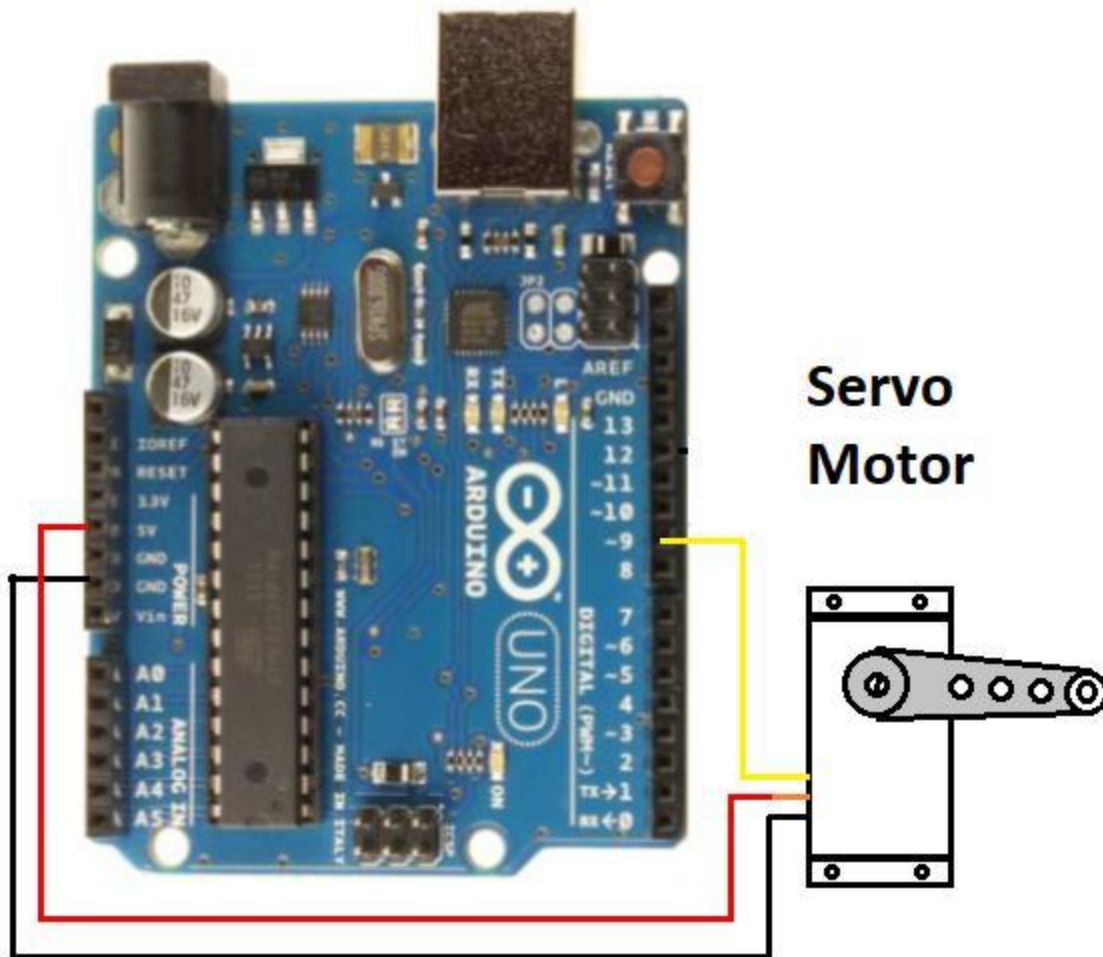
1.4.4. Vai trò trong hệ thống

Module dùng để đọc UID thẻ và gửi về ESP32.

1.5. Servo motor

1.5.1. Giới thiệu

Servo là động cơ điều khiển góc quay chính xác.



1.5.2. Nguyên lý hoạt động

Servo nhận tín hiệu PWM để điều khiển góc quay.

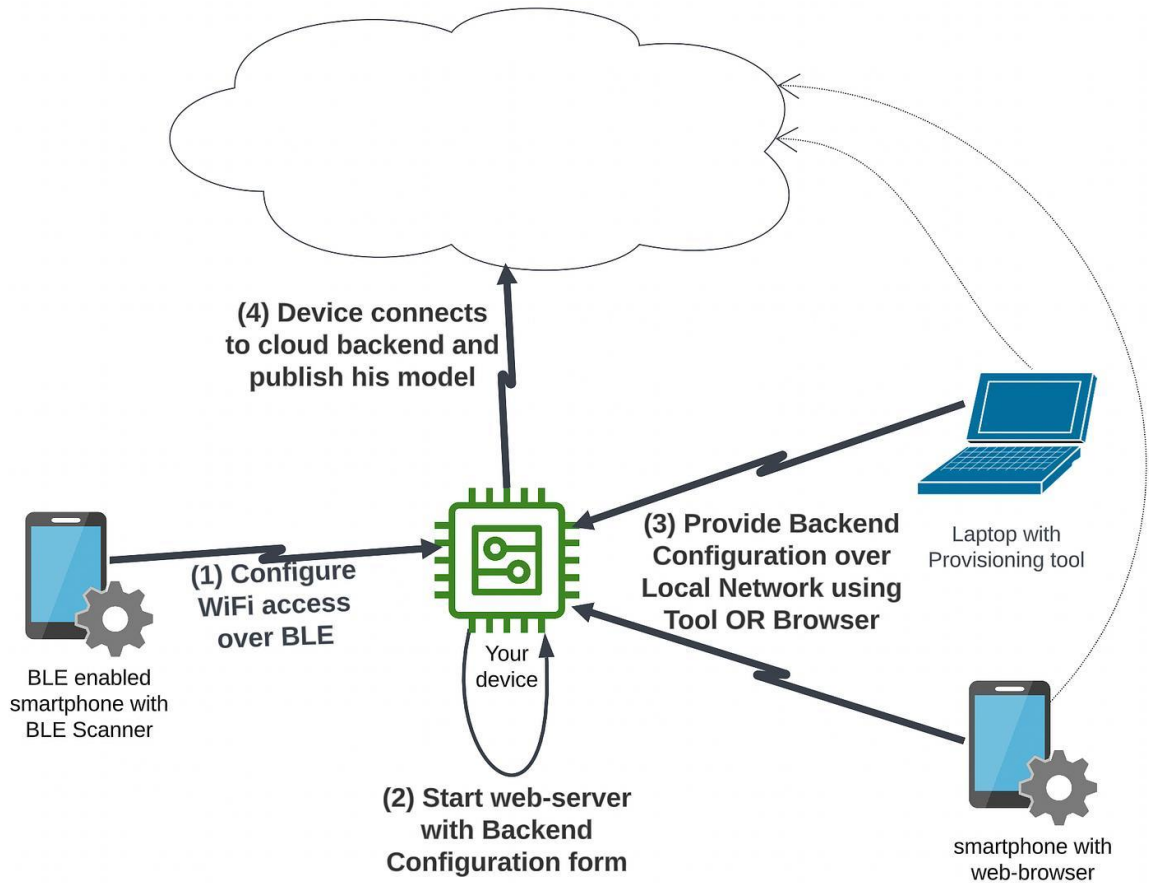
1.5.3. Vai trò trong đề tài

Servo được sử dụng để mô phỏng cơ cấu đóng/mở cửa.

1.6. Wi-Fi trong hệ thống

1.6.1. Giới thiệu

Wi-Fi là công nghệ mạng không dây cho phép kết nối Internet.



1.6.2. Vai trò trong hệ thống

Wi-Fi giúp ESP32 giao tiếp với Telegram để xác thực.

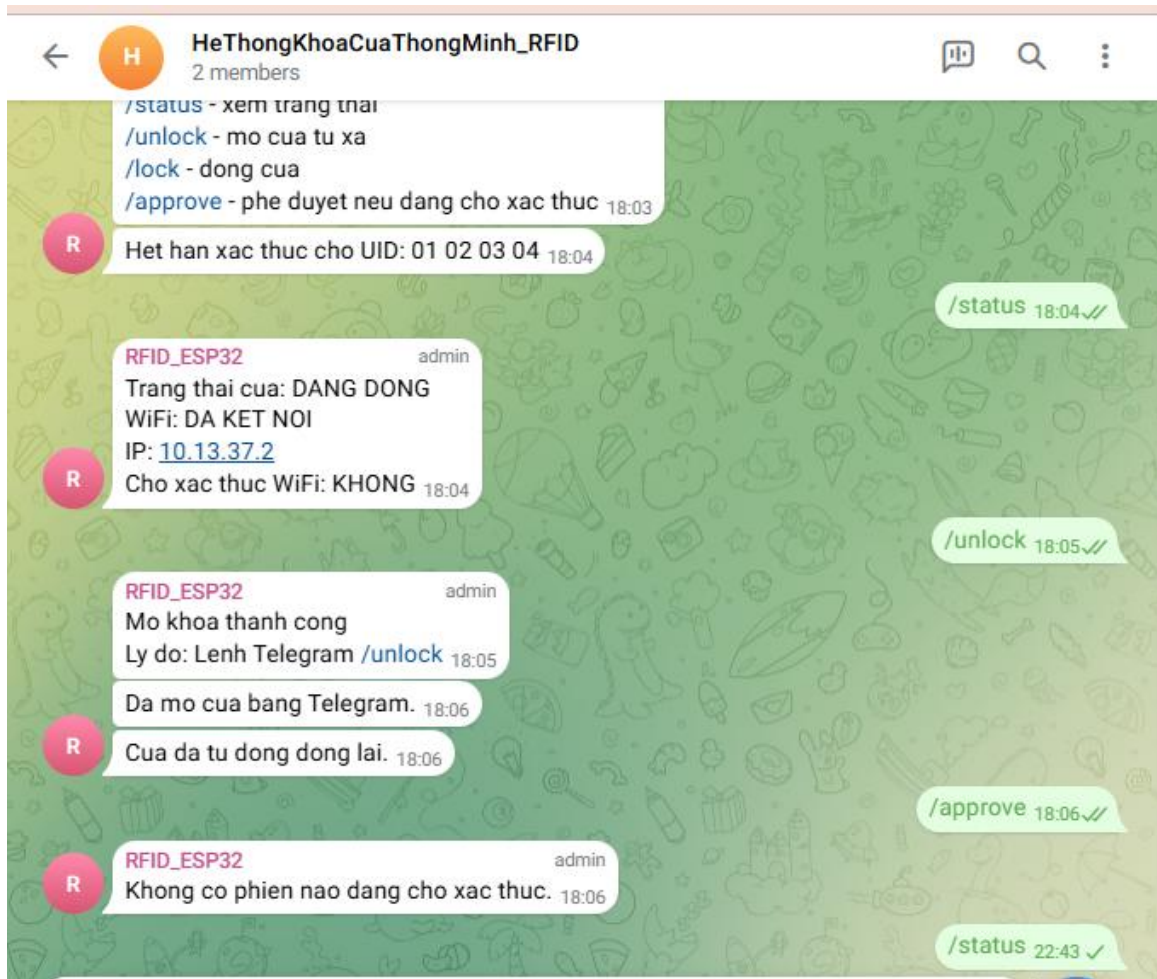
1.6.3. Quy trình xác thực

- Quẹt thẻ RFID
- Gửi thông báo lên Telegram
- Người dùng xác nhận
- Mở cửa nếu hợp lệ

1.7. Telegram Bot

1.7.1. Giới thiệu

Telegram Bot là công cụ xây dựng ứng dụng tự động trên Telegram.



1.7.2. Chức năng trong hệ thống

- Gửi thông báo
- Xác thực người dùng
- Điều khiển từ xa

1.7.3. Vai trò trong đề tài

Telegram đóng vai trò lớp xác thực thứ hai.

1.8. LED và buzzer

1.8.1. Giới thiệu

LED và buzzer là các thiết bị hiển thị trạng thái.

1.8.2. Vai trò trong hệ thống

- LED xanh: mở cửa
- LED đỏ: lỗi
- Buzzer: cảnh báo

1.9. Kết luận chương

Trong chương này, các cơ sở lý thuyết liên quan đến đề tài đã được trình bày một cách hệ thống. Các thành phần như ESP32, RFID, RC522, servo, Wi-Fi và Telegram Bot đóng vai trò quan trọng trong việc xây dựng hệ thống khóa cửa thông minh với cơ chế xác thực hai lớp.

Những kiến thức này là nền tảng để tiến hành thiết kế và triển khai hệ thống trong chương tiếp theo.

CHƯƠNG 2: PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu bài toán

2.1.1. Mục tiêu hệ thống

Mục tiêu của hệ thống là xây dựng một mô hình khóa cửa thông minh có khả năng kiểm soát truy cập bằng thẻ RFID, đồng thời tăng cường bảo mật thông qua cơ chế xác thực bổ sung bằng Telegram qua kết nối Wi-Fi.

2.1.2. Các yêu cầu cơ bản

Hệ thống cần đáp ứng các yêu cầu sau:

- Nhận diện thẻ RFID thông qua module RC522
- Kiểm tra UID của thẻ có hợp lệ hay không
- Không mở cửa ngay khi quét thẻ hợp lệ mà chuyển sang trạng thái chờ xác thực
- Gửi thông báo yêu cầu xác thực qua Telegram
- Cho phép người dùng xác nhận thông qua Telegram
- Mở cửa nếu xác thực thành công

- Tự động đóng cửa sau một khoảng thời gian
- Hủy yêu cầu nếu xác thực thất bại hoặc quá thời gian

2.2. Yêu cầu chức năng

2.2.1. Chức năng nhận diện thẻ RFID

Khi người dùng đưa thẻ vào vùng quét, module RFID đọc UID và gửi dữ liệu về ESP32. Bộ điều khiển tiến hành so sánh UID với danh sách UID hợp lệ đã được lưu trữ trong chương trình.

2.2.2. Chức năng xác thực hai lớp

Nếu UID hợp lệ, hệ thống không mở cửa ngay mà chuyển sang bước xác thực thứ hai. Trong bước này, người dùng cần gửi lệnh xác nhận (ví dụ: /approve) thông qua Telegram trong khoảng thời gian cho phép.

2.2.3. Chức năng mở và đóng cửa

Khi xác thực thành công, ESP32 điều khiển servo quay đến góc mở cửa. Sau một khoảng thời gian định trước, servo sẽ quay về vị trí ban đầu để đóng cửa.

2.2.4. Chức năng hiển thị trạng thái

Trạng thái hệ thống được hiển thị thông qua LED và buzzer:

- LED đỏ: báo trạng thái chờ xác thực hoặc truy cập bị từ chối
- LED xanh: báo trạng thái mở cửa thành công
- Buzzer: phát tín hiệu âm thanh tương ứng với từng trạng thái

2.2.5. Chức năng giám sát và điều khiển từ xa

Thông qua Telegram, người dùng có thể:

- Xem trạng thái hệ thống
- Phê duyệt mở cửa
- Mở cửa từ xa
- Nhận thông báo từ hệ thống

2.3. Yêu cầu phi chức năng

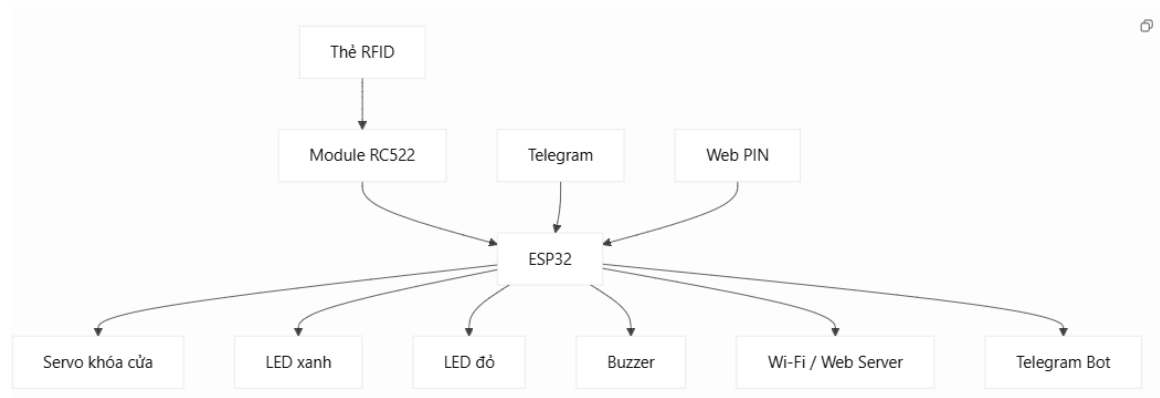
Hệ thống cần đảm bảo các yêu cầu sau:

- Hoạt động ổn định trong môi trường mô phỏng Wokwi
- Thời gian phản hồi nhanh
- Dễ sử dụng và dễ quan sát trạng thái
- Cấu trúc đơn giản, dễ mở rộng
- Dễ bảo trì và thay đổi danh sách UID

2.4. Sơ đồ khối hệ thống

Hệ thống được thiết kế gồm các khối chức năng chính:

- Khối đầu vào:
 - Thẻ RFID
- Khối xử lý trung tâm:
 - ESP32
- Khối truyền thông:
 - Wi-Fi
 - Telegram Bot
- Khối đầu ra:
 - Servo (mô phỏng khóa cửa)
 - LED xanh
 - LED đỏ
 - Buzzer



2.5. Nguyên lý hoạt động của hệ thống

Nguyên lý hoạt động của hệ thống được mô tả như sau:

- ESP32 khởi động và kết nối Wi-Fi
- Hệ thống ở trạng thái chờ quét thẻ RFID
- Khi người dùng quét thẻ:
 - RC522 đọc UID và gửi về ESP32
- ESP32 kiểm tra UID:
 - Nếu không hợp lệ → từ chối truy cập, bật LED đỏ và phát cảnh báo
 - Nếu hợp lệ → chuyển sang trạng thái chờ xác thực
- Hệ thống gửi thông báo qua Telegram yêu cầu xác nhận
- Người dùng gửi lệnh xác nhận (/approve)
- Nếu xác thực thành công:
 - Servo mở cửa
 - LED xanh sáng
 - Gửi thông báo thành công
- Sau thời gian quy định:
 - Cửa tự động đóng
- Nếu quá thời gian chờ:
 - Hệ thống hủy yêu cầu và gửi thông báo hết hiệu lực

2.6. Thiết kế sơ đồ kết nối phần cứng

2.6.1. Kết nối RFID RC522 với ESP32

RC522	ESP32
SDA	GPIO 5
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19
RST	GPIO 22
3.3V	3V3

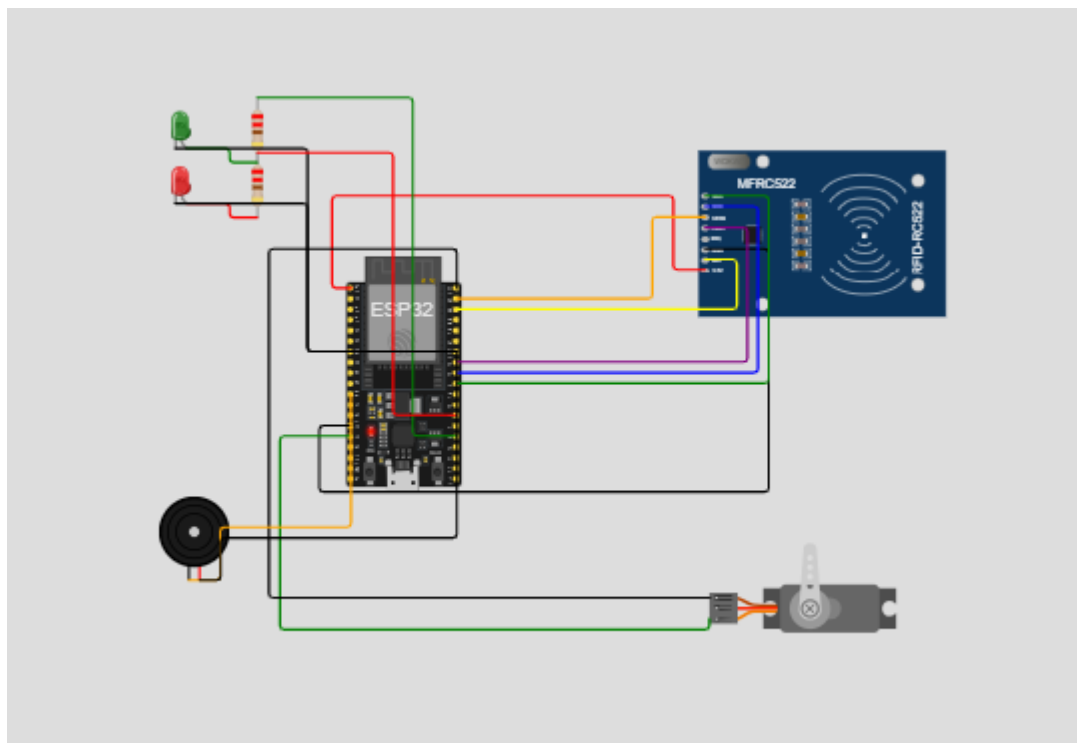
RC522	ESP32
GND	GND

2.6.2. Kết nối servo

Servo	ESP32
PWM	GPIO 13
VCC	VIN
GND	GND

2.6.3. Kết nối LED và buzzer

Thiết bị	ESP32
LED xanh	GPIO 2
LED đỏ	GPIO 4
Buzzer	GPIO 27



2.7. Thiết kế phần mềm

Phần mềm hệ thống được xây dựng theo mô hình xử lý tuần tự trong vòng lặp chính của ESP32. Các chức năng chính gồm:

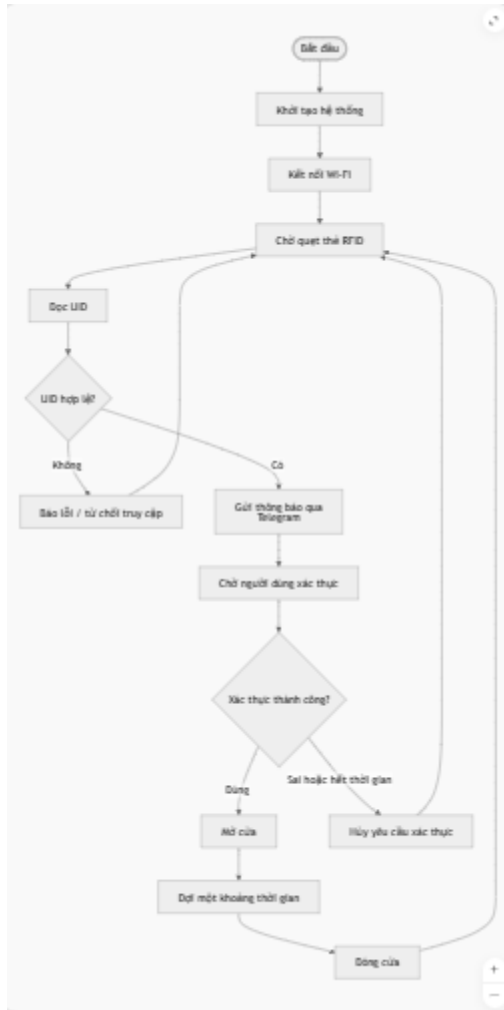
- Khởi tạo hệ thống và kết nối Wi-Fi
- Đọc dữ liệu từ RFID
- Gửi và nhận dữ liệu từ Telegram
- Xử lý xác thực hai lớp
- Kiểm tra thời gian xác thực
- Điều khiển servo, LED và buzzer

Các chức năng được tổ chức thành các hàm riêng biệt nhằm thuận tiện cho việc phát triển và bảo trì.

2.8. Lưu đồ thuật toán

Thuật toán của hệ thống có thể mô tả theo trình tự sau:

- Bắt đầu
- Khởi tạo hệ thống
- Kết nối Wi-Fi
- Chờ quét thẻ RFID
- Đọc UID
- Kiểm tra UID hợp lệ?
 - Nếu không → báo lỗi → quay lại
 - Nếu có → gửi Telegram → chờ xác thực
- Người dùng xác nhận qua Telegram
- Kiểm tra xác thực:
 - Nếu đúng → mở cửa → đóng cửa sau thời gian
 - Nếu sai hoặc hết thời gian → hủy
- Quay lại trạng thái chờ



2.9. Kết luận chương

Trong chương này, yêu cầu bài toán và phương án thiết kế hệ thống đã được phân tích và xây dựng một cách rõ ràng. Hệ thống được thiết kế theo mô hình xác thực hai lớp gồm RFID và Telegram thông qua Wi-Fi, trong đó ESP32 đóng vai trò trung tâm điều khiển.

Các thành phần phần cứng và phần mềm đã được xác định cụ thể, làm cơ sở cho việc triển khai và lập trình hệ thống trong chương tiếp theo

CHƯƠNG 3: THI CÔNG HỆ THỐNG

3.1. Giới thiệu chương

Sau khi đã phân tích yêu cầu và thiết kế hệ thống ở chương trước, chương này trình bày quá trình triển khai mô hình khóa cửa thông minh dựa trên ESP32 và RFID, tích hợp điều khiển từ xa qua Wi-Fi bằng Telegram. Nội dung chương tập trung vào hai phần chính là thi công phần cứng và xây dựng phần mềm điều khiển cho hệ thống.

Bên cạnh đó, chương này cũng mô tả quá trình mô phỏng hệ thống trên nền tảng Wokwi. Do đặc điểm của môi trường mô phỏng, việc truy cập web server để thực hiện xác thực không được sử dụng làm hướng thực nghiệm chính. Vì vậy, hệ thống được hoàn thiện theo hướng dùng RFID để nhận diện thẻ, sau đó gửi thông báo chờ xác thực qua Telegram và cho phép người dùng điều khiển, xác nhận từ xa thông qua Wi-Fi.

3.2. Danh sách linh kiện sử dụng

Bảng 3.1. Danh sách linh kiện sử dụng trong hệ thống

STT	Tên linh kiện	Số lượng	Chức năng
1	ESP32 DevKit	1	Bộ điều khiển trung tâm
2	Module RFID RC522	1	Đọc UID thẻ RFID
3	Thẻ RFID	1 hoặc nhiều	Định danh người dùng
4	Servo motor	1	Mô phỏng khóa cửa
5	LED xanh	1	Báo mở cửa
6	LED đỏ	1	Báo trạng thái chờ
7	Buzzer	1	Phát âm thanh
8	Điện trở 220Ω	2	Hạn dòng LED
9	Dây nối	Nhiều	Kết nối

3.3. Thi công phần cứng

3.3.1. Vai trò các khối phần cứng

Hệ thống phần cứng được xây dựng từ các khối chức năng chính như sau:

- ESP32 là bộ điều khiển trung tâm, có nhiệm vụ đọc dữ liệu từ RFID, xử lý logic xác thực, điều khiển servo, LED, buzzer và giao tiếp với Telegram qua Wi-Fi.
- RC522 là module đọc thẻ RFID, được dùng để nhận diện UID của người dùng.
- Servo motor được sử dụng để mô phỏng trạng thái đóng và mở cửa cửa.
- LED xanh dùng để hiển thị trạng thái cửa đã mở thành công.
- LED đỏ dùng để biểu thị trạng thái hệ thống đang chờ xác thực hoặc đang ở chế độ cảnh báo.
- Buzzer phát tín hiệu âm thanh nhằm hỗ trợ người dùng nhận biết các trạng thái hoạt động của hệ thống.

Nhờ sự phối hợp giữa các thành phần trên, hệ thống có thể vừa thực hiện chức năng mở khóa, vừa hiển thị trạng thái một cách trực quan.

3.3.2. Sơ đồ kết nối phần cứng

Trong mô hình thực hiện, các linh kiện được kết nối với ESP32 như sau:

Kết nối module RFID RC522 với ESP32

- SDA nối với GPIO 5
- SCK nối với GPIO 18
- MOSI nối với GPIO 23
- MISO nối với GPIO 19

- RST nối với GPIO 22
- 3.3V nối với chân 3V3
- GND nối với GND

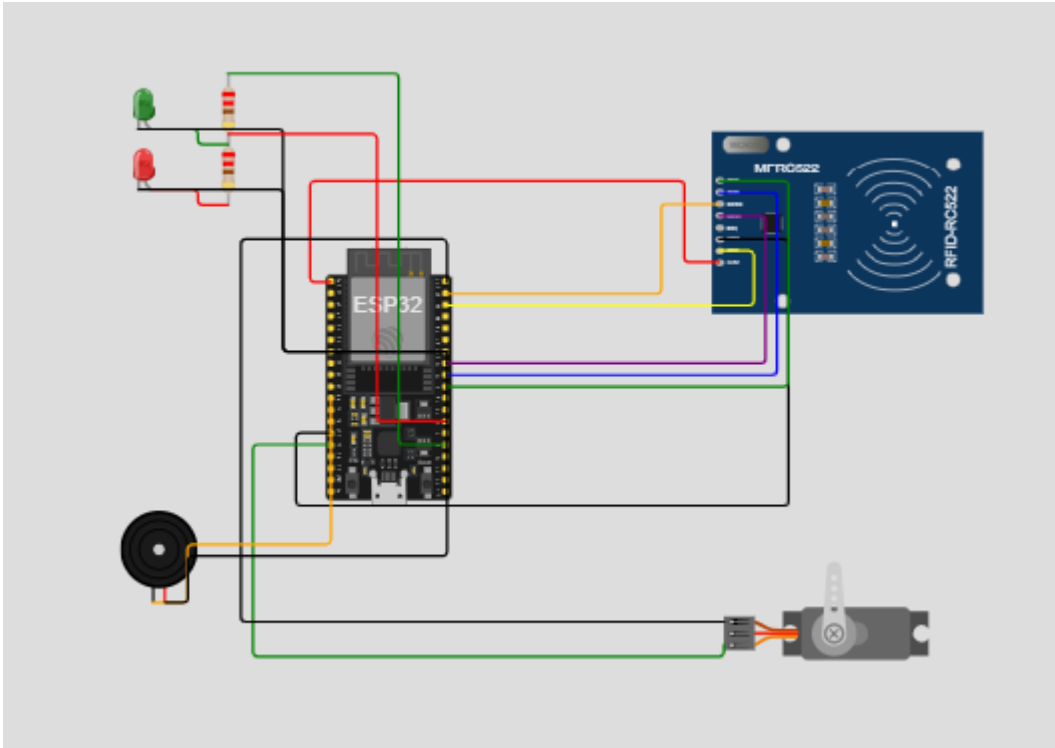
Kết nối servo

- Chân PWM nối với GPIO 13
 - Chân nguồn nối với VIN
 - Chân GND nối với GND
-

Kết nối LED và buzzer

- LED xanh nối với GPIO 2
 - LED đỏ nối với GPIO 4
 - Buzzer nối với GPIO 27
-

Thiết bị	Chân thiết bị	Chân ESP32
RC522	SDA	GPIO 5
RC522	SCK	GPIO 18
RC522	MOSI	GPIO 23
RC522	MISO	GPIO 19
RC522	RST	GPIO 22
Servo	PWM	GPIO 13
LED xanh	Anode	GPIO 2
LED đỏ	Anode	GPIO 4
Buzzer	Signal	GPIO 27



3.3.3. Mô tả hoạt động phần cứng

Khi hệ thống được cấp nguồn, ESP32 tiến hành khởi tạo các thiết bị phần cứng và kết nối với mạng Wi-Fi. Sau đó, RC522 liên tục quét thẻ RFID để phát hiện UID của người dùng.

Khi người dùng bấm TAP trong môi trường Wokwi, hệ thống đọc UID của thẻ và tiến hành kiểm tra. Nếu thẻ hợp lệ, buzzer phát một tiếng báo hiệu, LED đỏ sáng và hệ thống chuyển sang trạng thái chờ xác thực. Trong thời gian này, servo chưa mở cửa mà chờ xác nhận bổ sung từ Telegram.

Khi người dùng xác nhận thành công thông qua Telegram, servo quay sang góc mở khóa, LED xanh sáng và hệ thống gửi thông báo mở khóa thành công. Sau một khoảng thời gian nhất định, servo tự động quay về vị trí ban đầu để mô phỏng việc đóng cửa.

Nếu UID không hợp lệ hoặc hết thời gian xác thực, hệ thống sẽ không mở cửa. Khi đó LED đỏ và buzzer được sử dụng để biểu thị trạng thái cảnh báo hoặc từ chối truy cập.

3.4. Thi công phần mềm

3.4.1. Môi trường phát triển

Phần mềm điều khiển hệ thống được xây dựng bằng ngôn ngữ C++ trên nền tảng Arduino dành cho ESP32. Quá trình viết chương trình và mô phỏng được thực hiện bằng PlatformIO kết hợp với Wokwi.

Các thư viện chính được sử dụng trong chương trình gồm:

- SPI.h
- MFRC522.h
- ESP32Servo.h
- WiFi.h
- WebServer.h
- WiFiClientSecure.h
- UniversalTelegramBot.h
- ArduinoJson.h

Các thư viện này giúp hỗ trợ giao tiếp với module RFID, điều khiển servo, quản lý kết nối Wi-Fi và giao tiếp với Telegram Bot.

3.4.2. Cấu trúc chương trình

Chương trình được tổ chức thành nhiều nhóm chức năng riêng biệt để thuận tiện cho việc phát triển và bảo trì.

Nhóm cấu hình

- cấu hình Wi-Fi
- cấu hình Telegram
- cấu hình chân GPIO
- cấu hình thông số khóa

Nhóm xử lý RFID

- đọc UID

- kiểm tra UID
- khởi tạo trạng thái chờ xác thực

Nhóm xử lý Telegram

- gửi thông báo trạng thái
- nhận và xử lý lệnh /status, /unlock, /lock, /approve

Nhóm điều khiển phần cứng

- servo
- LED
- buzzer

Nhóm xử lý thời gian

- kiểm tra thời gian chờ xác thực
- hủy UID hết hiệu lực
- tự động đóng cửa sau khi mở

Cấu trúc này giúp hệ thống hoạt động rõ ràng theo từng chức năng, giảm xung đột giữa các trạng thái trong quá trình mô phỏng.

3.4.3. Các hàm chính

Một số hàm chính trong chương trình bao gồm:

- `getUIDString()`: chuyển UID của thẻ RFID từ dạng byte sang chuỗi ký tự.
- `isAuthorizedCard()`: kiểm tra thẻ RFID có nằm trong danh sách hợp lệ hay không.
- `lockDoor()`: đóng cửa và đưa servo về vị trí khóa.
- `unlockDoor()`: mở cửa và gửi thông báo thành công.
- `denyAccess()`: từ chối truy cập và kích hoạt cảnh báo.
- `startWifiAuth()`: khởi tạo trạng thái chờ xác thực sau khi quét thẻ hợp lệ.
- `handleTelegramMessages()`: nhận và xử lý các lệnh từ Telegram.
- `handleRFID()`: xử lý sự kiện quét thẻ RFID.
- `pollTelegram()`: kiểm tra và cập nhật tin nhắn Telegram mới.

Một số đoạn mã quan trọng được trình bày dưới đây để minh họa cho nguyên lý hoạt động của hệ thống.

Đoạn mã 3.1. Hàm đọc UID của thẻ RFID

```
Windsurf: Refactor | Explain | X
3  ✓ String getUIDString(MFRC522::Uid* uid) {
4      String result = "";
5  ✓  for ([byte i = 0; i < uid->size; i++]) {
6      if (uid->uidByte[i] < 0x10) result += "0";
7      result += String(uid->uidByte[i], HEX);
8      if (i < uid->size - 1) result += " ";
9  }
10     result.toUpperCase();
11     return result;
12 }
13
```

Hàm này dùng để chuyển dữ liệu UID nhận từ RC522 sang dạng chuỗi để hệ thống dễ so sánh với danh sách các thẻ đã được cấp quyền.

Đoạn mã 3.2. Hàm bắt đầu trạng thái chờ xác thực

```

void startWifiAuth(const String& uid) {
    pendingWifiAuth = true;
    authStartedAt = millis();
    currentUid = uid;

    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED, HIGH);
    beepWait();

    String ip = WiFi.localIP().toString();

    Serial.println("RFID hop le. Dang cho xac thuc Wi-Fi...");
    Serial.print("UID dang cho: ");
    Serial.println(uid);
    Serial.print("Mo trinh duyet den: http://");
    Serial.println(ip);

    String msg = "The RFID hop le: " + uid +
        "\nDang cho xac thuc Wi-Fi/PIN trong 30 giay." +
        "\nIP ESP32: http://" + ip;
    sendTelegramMessage(msg);
}

```

Khi người dùng quét thẻ hợp lệ, hệ thống không mở cửa ngay mà chuyển sang trạng thái chờ xác thực. Đèn đỏ sáng, buzzer phát tín hiệu và Telegram nhận được thông báo về UID đang chờ xác thực.

Đoạn mã 3.3. Hàm xử lý quét thẻ RFID

```

void handleRFID() {
    if (millis() - lastCardReadAt < RFID_COOLDOWN_MS) return;
    if (pendingWifiAuth) return;
    if (!rfid.PICC_IsNewCardPresent()) return;
    Serial.println("Card present detected");
    if (!rfid.PICC_ReadCardSerial()) {
        Serial.println("Card present but UID read failed");
        return;
    }
    lastCardReadAt = millis();

    String uid = getUIDString(&rfid.uid);
    Serial.print("Card detected. UID: ");
    Serial.println(uid);

    if (isAuthorizedCard(uid)) {
        startWifiAuth(uid);
    } else {
        denyAccess("The RFID không hợp lệ: " + uid);
    }

    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
}

```

Đoạn mã này thể hiện quá trình đọc thẻ, kiểm tra UID và quyết định có tạo trạng thái chờ xác thực hay từ chối truy cập.

3.4.4. Nguyên lý hoạt động phần mềm

Phần mềm hoạt động dựa trên vòng lặp chính loop() của ESP32. Trong vòng lặp này, hệ thống liên tục thực hiện các tác vụ sau:

- quét thẻ RFID
- nhận và xử lý tin nhắn từ Telegram
- kiểm tra xem phiên xác thực có còn hiệu lực hay không
- kiểm tra trạng thái cửa để tự động đóng lại sau thời gian quy định

Với cách tổ chức này, hệ thống có thể hoạt động liên tục, cập nhật trạng thái theo thời gian thực và xử lý đồng thời nhiều chức năng như RFID, Telegram và điều khiển servo.

3.5. Cơ chế xác thực và điều khiển hệ thống qua Wi-Fi bằng Telegram

Hệ thống khóa cửa thông minh trong đề tài được xây dựng theo cơ chế xác thực hai lớp, trong đó RFID đóng vai trò nhận diện ban đầu, còn Wi-Fi thông qua Telegram đóng vai trò xác thực bổ sung và điều khiển từ xa.

Ở lớp xác thực thứ nhất, người dùng quét thẻ RFID để hệ thống đọc UID. Nếu UID không nằm trong danh sách hợp lệ, hệ thống sẽ từ chối truy cập ngay lập tức, đồng thời phát cảnh báo bằng LED đỏ, buzzer và gửi thông báo lên Telegram.

Nếu UID hợp lệ, hệ thống chuyển sang trạng thái chờ xác thực. Khi đó, buzzer phát một tiếng ngắn, LED đỏ sáng và ESP32 gửi thông báo lên Telegram rằng UID tương ứng đang chờ xác nhận. Phiên xác thực này chỉ có hiệu lực trong vòng 30 giây.

Trong khoảng thời gian đó, người dùng có thể sử dụng Telegram để điều khiển hệ thống thông qua Wi-Fi. Các lệnh chính bao gồm:

- /status: xem trạng thái hiện tại của cửa và hệ thống
- /approve: phê duyệt mở cửa cho UID đang chờ xác thực
- /unlock: mở cửa từ xa
- /lock: đóng cửa

Như vậy, Telegram không chỉ là công cụ thông báo mà còn là phương tiện điều khiển từ xa chính của hệ thống thông qua Wi-Fi. Đây là điểm phù hợp với định hướng xây dựng một hệ thống khóa cửa thông minh có khả năng giám sát và điều khiển từ xa.

Nếu hết 30 giây mà không có xác nhận hợp lệ, hệ thống sẽ tự động hủy UID đang chờ, tắt trạng thái chờ và gửi thông báo lên

Telegram rằng UID đó đã hết hiệu lực. Cơ chế này giúp tăng tính bảo mật và tránh duy trì phiên truy cập quá lâu.

3.6. Mô phỏng hệ thống trên Wokwi

Trong quá trình thực hiện đề tài, Wokwi được sử dụng để mô phỏng toàn bộ hệ thống trước khi triển khai trên phần cứng thật. Môi trường này cho phép mô phỏng ESP32, RC522, servo, LED, buzzer và quá trình quét thẻ RFID.

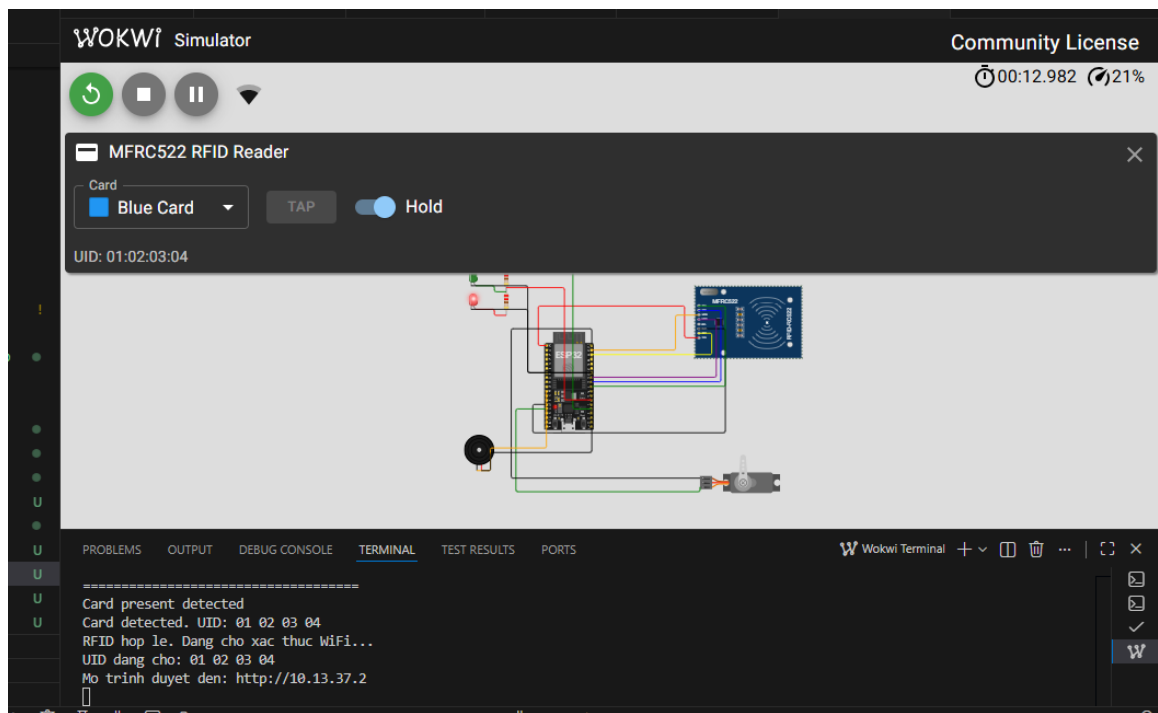
Ưu điểm của việc mô phỏng trên Wokwi gồm:

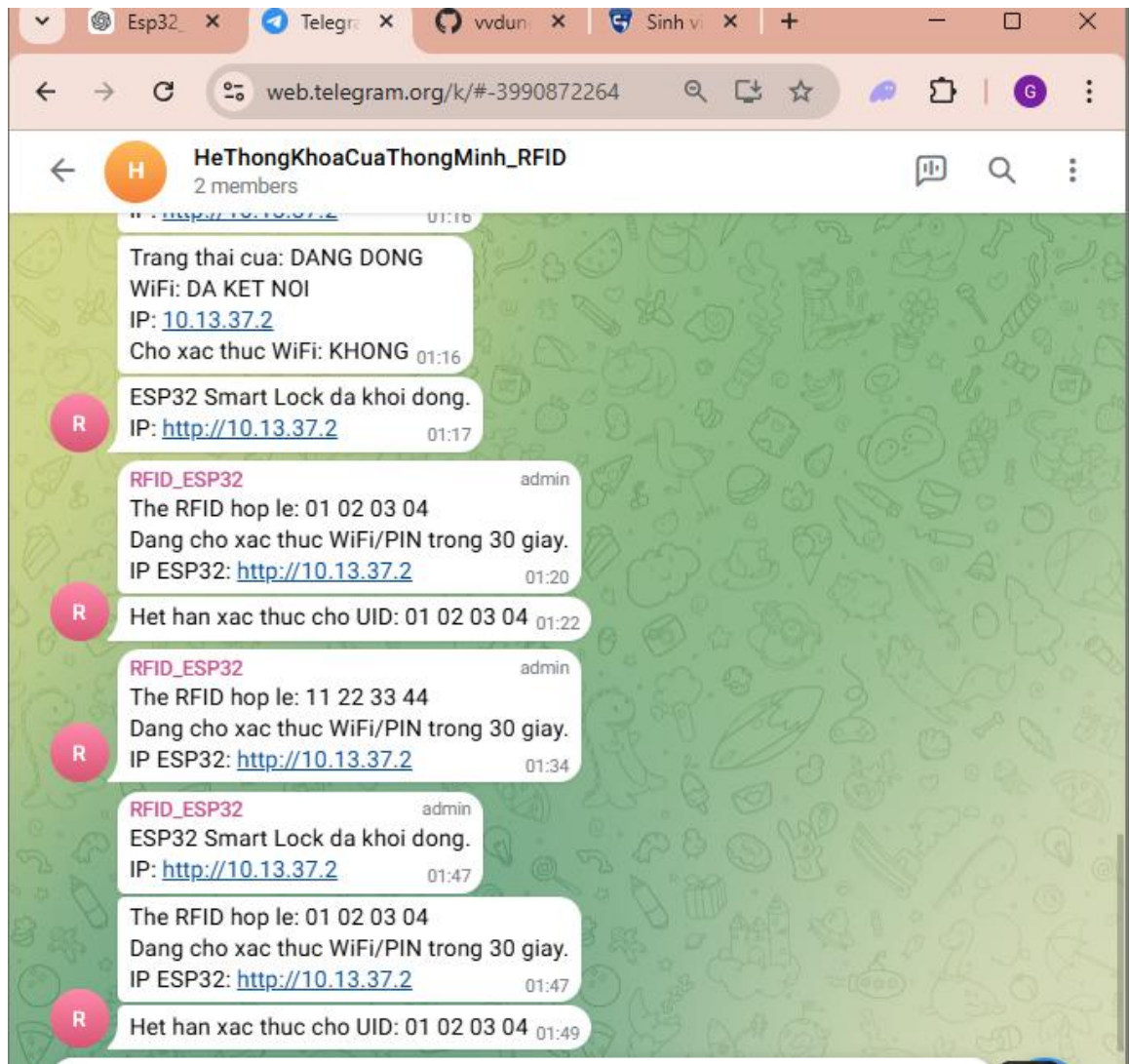
- không cần lắp ráp phần cứng thật trong giai đoạn đầu
- dễ kiểm tra lỗi kết nối và lỗi logic chương trình
- thuận tiện cho việc quan sát trạng thái của từng linh kiện
- dễ chụp ảnh kết quả để đưa vào báo cáo

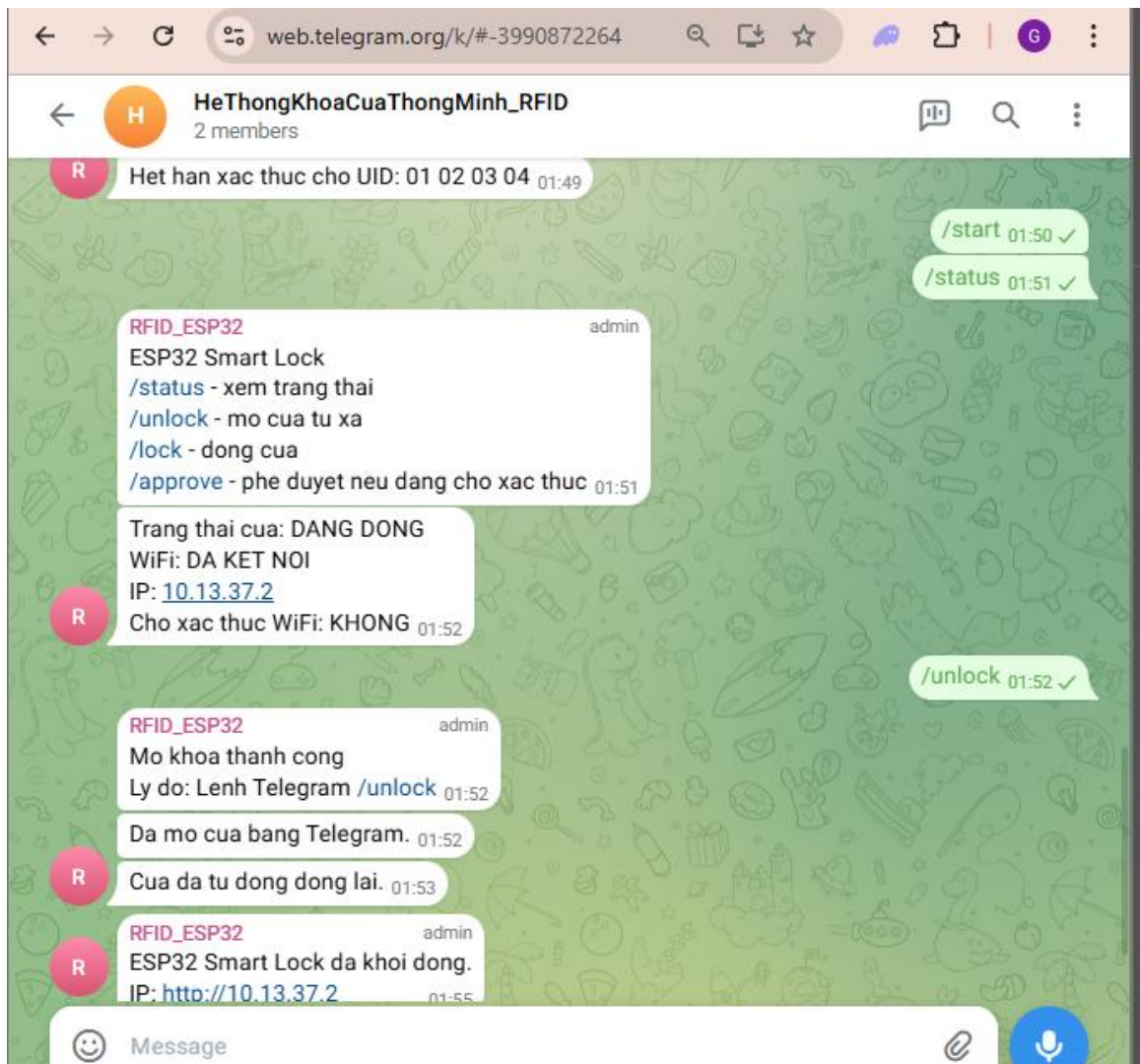
Trong mô hình thực nghiệm hiện tại, người dùng bấm TAP để mô phỏng hành động quét thẻ. Khi thẻ hợp lệ được quét, hệ thống sẽ:

- đọc UID
- phát tín hiệu âm thanh
- bật LED đỏ
- gửi thông báo lên Telegram
- chuyển sang trạng thái chờ xác thực

Do giới hạn của môi trường mô phỏng, việc truy cập trực tiếp vào web server để nhập mã PIN không được sử dụng làm hướng kiểm thử chính. Vì vậy, kịch bản thực nghiệm của đề tài tập trung vào việc xác nhận và điều khiển thông qua Telegram. Nếu trong thời gian cho phép không có xác nhận hợp lệ, hệ thống sẽ thông báo rằng UID đã hết hiệu lực.







3.7. Khó khăn trong quá trình thực hiện

Trong quá trình xây dựng hệ thống, một số khó khăn đã được ghi nhận như sau:

- việc kết nối và duy trì Wi-Fi trong môi trường mô phỏng cần được theo dõi cẩn thận
- việc đồng bộ giữa trạng thái RFID, Telegram, LED, buzzer và servo đòi hỏi logic chương trình phải rõ ràng
- việc kiểm soát thời gian hiệu lực của UID cần chính xác để tránh lỗi xác thực
- môi trường Wokwi không thuận tiện cho việc khai thác web server như trong phần cứng thật
- việc biểu diễn tín hiệu đèn và âm thanh trong mô phỏng cần được tinh chỉnh để người dùng dễ quan sát

Tuy nhiên, các khó khăn trên đã được xử lý bằng cách tối ưu chương trình và chuyển trọng tâm thực nghiệm sang xác thực, điều khiển qua Telegram.

3.8. Kết luận chương

Chương này đã trình bày quá trình thi công hệ thống khóa cửa thông minh dựa trên ESP32 và RFID từ phần cứng đến phần mềm. Hệ thống sử dụng RFID để nhận diện ban đầu và sử dụng Wi-Fi thông qua Telegram để thực hiện chức năng điều khiển, xác nhận và giám sát từ xa. Mặc dù môi trường mô phỏng Wokwi còn hạn chế trong việc sử dụng web server làm kịch bản thực nghiệm chính, hệ thống vẫn hoạt động đúng hướng thiết kế và đáp ứng được yêu cầu cốt lõi của đề tài. Đây là cơ sở quan trọng để tiến hành kiểm thử và đánh giá kết quả hoạt động ở chương tiếp theo.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Giới thiệu chương

Chương này trình bày quá trình kiểm thử hệ thống khóa cửa thông minh dựa trên ESP32 và RFID trong môi trường mô phỏng Wokwi, đồng thời đánh giá khả năng hoạt động của hệ thống thông qua các tình huống thực nghiệm cụ thể. Nội dung chương tập trung vào việc xác minh tính đúng đắn của các chức năng chính như nhận diện thẻ RFID, xác thực thông qua Telegram, điều khiển đóng mở cửa từ xa, xử lý cảnh báo và quản lý thời gian hiệu lực của phiên xác thực.

4.2. Môi trường thực nghiệm

Việc thực nghiệm hệ thống được tiến hành trên nền tảng mô phỏng Wokwi, sử dụng ESP32, module RFID RC522, servo, LED, buzzer và Telegram Bot. Trong quá trình mô phỏng, hệ thống hoạt động theo đúng nguyên lý đã thiết kế:

- người dùng bấm TAP để mô phỏng hành động quét thẻ RFID
- ESP32 đọc UID của thẻ và kiểm tra tính hợp lệ
- nếu thẻ hợp lệ, hệ thống chuyển sang trạng thái chờ xác thực
- Telegram được sử dụng để gửi thông báo và điều khiển từ xa thông qua Wi-Fi

Do môi trường Wokwi không thuận tiện cho việc sử dụng trực tiếp web server như một kịch bản thực nghiệm chính, phần kiểm thử của đề tài tập trung vào cơ chế điều khiển và xác thực qua Telegram.

4.3. Các trường hợp kiểm thử

Để đánh giá hệ thống, đề tài xây dựng một số tình huống kiểm thử tiêu biểu, phản ánh các trạng thái chính có thể xảy ra trong quá trình sử dụng.

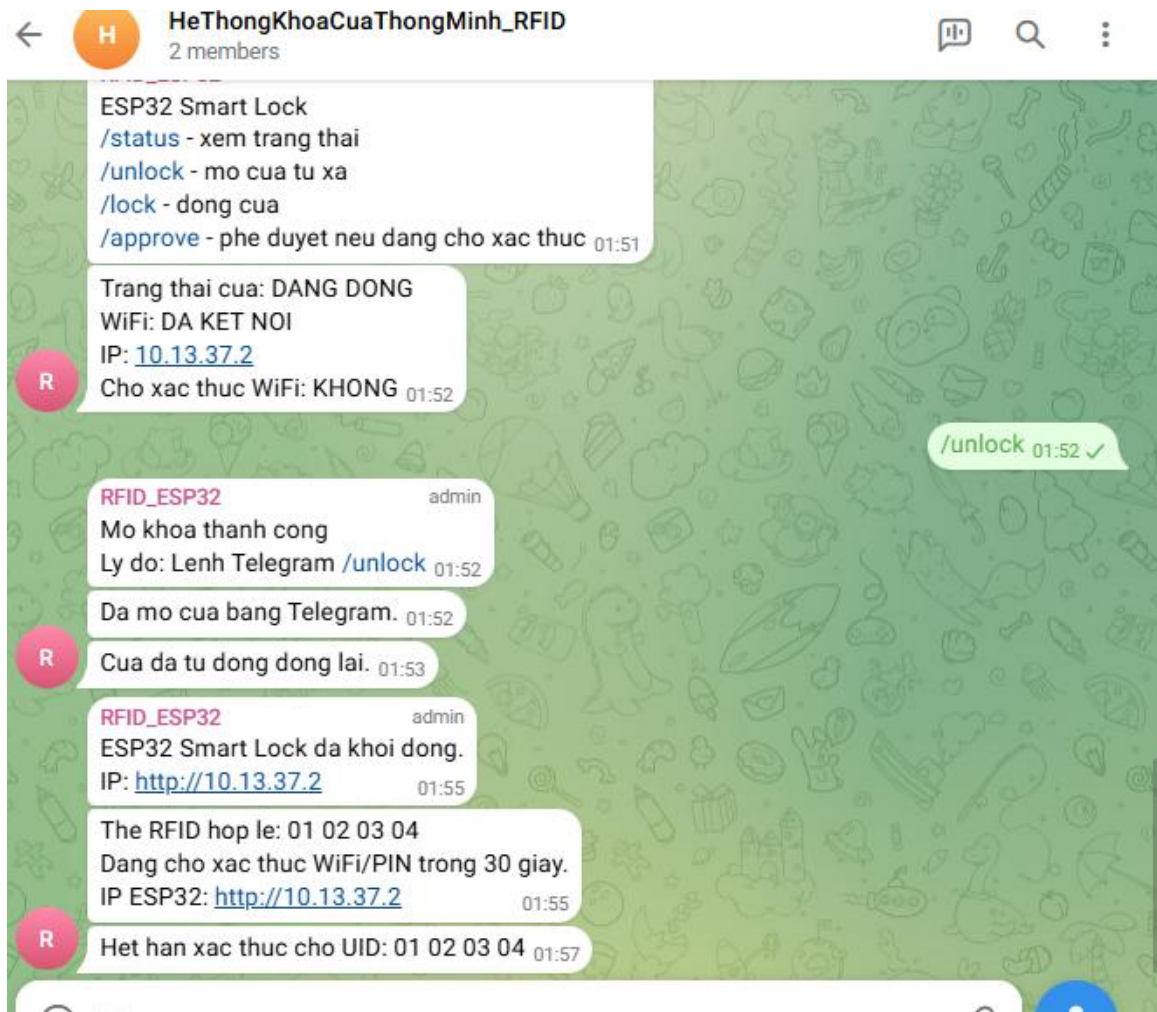
4.3.4. Trường hợp 4: UID hết hiệu lực sau 30 giây

Trong trường hợp này, người dùng quét thẻ hợp lệ nhưng không thực hiện xác nhận qua Telegram trong khoảng thời gian 30 giây.

Kết quả thu được:

-
- hệ thống duy trì trạng thái chờ xác thực trong 30 giây
 - sau khi hết thời gian, trạng thái chờ bị hủy
 - LED đỏ tắt
 - Telegram gửi thông báo rằng UID đã hết hiệu lực
 - hệ thống quay trở lại trạng thái sẵn sàng ban đầu
-

Kết quả thực nghiệm cho thấy hệ thống quản lý thời gian hiệu lực của phiên xác thực đúng theo thiết kế, góp phần tăng cường bảo mật và hạn chế việc lợi dụng các phiên truy cập còn tồn đọng.



4.3.5. Trường hợp 5: Mở cửa từ xa bằng lệnh /unlock

Trong trường hợp này, người dùng không cần quét thẻ mà gửi trực tiếp lệnh /unlock trên Telegram.

Kết quả thu được:

- hệ thống nhận lệnh điều khiển từ Telegram
- servo quay sang vị trí mở cửa
- LED xanh sáng
- buzzer phát tín hiệu xác nhận
- Telegram gửi thông báo mở cửa thành công

Kết quả này cho thấy hệ thống có thể hoạt động như một khóa cửa thông minh có điều khiển từ xa qua Wi-Fi.

4.3.6. Trường hợp 6: Đóng cửa bằng lệnh /lock

Khi người dùng gửi lệnh /lock, hệ thống thực hiện:

- servo quay về vị trí khóa
- LED xanh tắt
- các trạng thái liên quan đến mở cửa được đặt lại
- Telegram gửi thông báo cửa đã đóng

Kết quả cho thấy chức năng đóng cửa từ xa hoạt động ổn định và phù hợp với yêu cầu thiết kế.

4.3.7. Trường hợp 7: Kiểm tra trạng thái bằng lệnh /status

Khi gửi lệnh /status, Telegram hiển thị các thông tin như:

- trạng thái cửa đang mở hay đóng
- trạng thái kết nối Wi-Fi
- địa chỉ IP của ESP32
- có đang chờ xác thực hay không
- UID đang chờ xác thực nếu có

Chức năng này hỗ trợ người dùng giám sát hệ thống từ xa một cách thuận tiện.

4.4. Bảng tổng hợp kết quả kiểm thử

STT	Tình huống kiểm thử	Kết quả mong đợi	Kết quả thực tế	Đánh giá
1	Quét thẻ hợp lệ	Bật đèn đỏ, phát tín hiệu, gửi Telegram, chờ xác thực	Đúng như mong đợi	Đạt
2	Quét thẻ không hợp lệ	Từ chối truy cập, cảnh báo, gửi Telegram	Đúng như mong đợi	Đạt
3	Gửi /approve khi đang chờ	Mở cửa, bật đèn xanh, gửi xác nhận	Đúng như mong đợi	Đạt
4	Không xác thực trong 30 giây	Hủy phiên chờ, báo UID hết hiệu lực	Đúng như mong đợi	Đạt
5	Gửi /unlock	Mở cửa từ xa	Đúng như mong đợi	Đạt
6	Gửi /lock	Đóng cửa	Đúng như mong đợi	Đạt
7	Gửi /status	Hiển thị trạng thái hệ thống	Đúng như mong đợi	Đạt

4.5. Đánh giá hệ thống

4.5.1. Ưu điểm

Qua quá trình thực nghiệm, hệ thống đạt được một số ưu điểm nổi bật như sau:

-
- Cấu trúc hệ thống đơn giản, dễ xây dựng và dễ mở rộng.
 - ESP32 đáp ứng tốt vai trò trung tâm điều khiển.
 - RFID giúp nhận diện người dùng nhanh chóng và thuận tiện.
 - Telegram hỗ trợ giám sát và điều khiển từ xa hiệu quả thông qua Wi-Fi.
 - Hệ thống có cơ chế xác thực hai lớp nên bảo mật cao hơn so với mô hình chỉ dùng RFID.
 - Mô phỏng trên Wokwi ổn định, trực quan và thuận tiện cho việc trình bày kết quả.
-

4.5.2. Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế như:

-
- Mô hình mới dừng ở mức mô phỏng, chưa triển khai đầy đủ trên phần cứng thực tế.
 - Việc sử dụng web server trong Wokwi chưa thuận tiện nên chưa được khai thác làm kịch bản thực nghiệm chính.
 - Hệ thống hiện mới dùng danh sách UID cố định, chưa có chức năng quản lý người dùng động.
 - Việc điều khiển qua Telegram phụ thuộc vào kết nối Wi-Fi ổn định.
 - Chưa tích hợp thêm các phương thức xác thực nâng cao như vân tay hoặc Bluetooth app.
-

4.5.3. Mức độ đáp ứng yêu cầu đề tài

So với yêu cầu đặt ra ban đầu, hệ thống đã đáp ứng được các nội dung cốt lõi của đề tài:

-
- Sử dụng ESP32 làm bộ điều khiển trung tâm
 - Sử dụng RFID để nhận diện người dùng

- Tích hợp xác thực và điều khiển qua Wi-Fi bằng Telegram
 - Mô phỏng cơ chế đóng mở cửa
 - Hiển thị và cảnh báo trạng thái hoạt động
-

Mặc dù chưa khai thác đầy đủ giao diện web trong thực nghiệm, hệ thống vẫn hoàn toàn phù hợp với định hướng đề tài là xây dựng mô hình khóa cửa thông minh dựa trên ESP32 và RFID, có xác thực bổ sung qua kết nối không dây.

4.6. Kết luận chương

Qua quá trình thực nghiệm, hệ thống đã hoạt động ổn định trong môi trường mô phỏng Wokwi và đáp ứng được các chức năng chính của đề tài. Các tình huống kiểm thử cho thấy hệ thống có thể nhận diện thẻ RFID, gửi thông báo chờ xác thực, điều khiển mở đóng cửa từ xa qua Telegram, xử lý trạng thái hết hiệu lực của UID và duy trì khả năng giám sát hệ thống. Đây là cơ sở quan trọng để rút ra kết luận chung và đề xuất hướng phát triển ở phần cuối của báo cáo.

CHƯƠNG 5: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau quá trình nghiên cứu, thiết kế, lập trình và mô phỏng, đề tài “**Hệ thống khóa cửa thông minh dựa trên ESP32 và RFID, tích hợp xác thực qua Wi-Fi**” đã được triển khai thành công trên môi trường Wokwi. Hệ thống đã đáp ứng được các mục tiêu cơ bản đã đề ra ban đầu, bao gồm nhận diện người dùng bằng thẻ RFID, điều khiển mô phỏng khóa cửa bằng servo, hiển thị trạng thái hoạt động bằng LED và buzzer, đồng thời hỗ trợ giám sát và điều khiển từ xa thông qua Telegram.

Trong quá trình thực hiện, hệ thống đã thể hiện được nguyên lý hoạt động của một mô hình khóa cửa thông minh có xác thực hai lớp. Ở lớp đầu tiên, thẻ RFID được sử dụng để nhận diện người dùng thông qua UID. Ở lớp thứ hai, hệ thống sử dụng kết nối Wi-Fi để gửi thông báo lên Telegram và chờ xác nhận trong một khoảng thời gian nhất định. Nếu người dùng xác nhận hợp lệ trong thời gian cho phép, cửa sẽ được mở; ngược lại, yêu cầu xác thực sẽ hết hiệu lực. Cơ chế này góp phần nâng cao tính bảo mật so với các mô hình chỉ sử dụng RFID đơn thuần.

Kết quả thực nghiệm cho thấy hệ thống hoạt động tương đối ổn định và đúng với mục tiêu thiết kế. Các chức năng chính như nhận diện thẻ RFID hợp lệ, từ chối thẻ không hợp lệ, gửi thông báo trạng thái lên Telegram, xác nhận bằng lệnh /approve, điều khiển đóng mở cửa bằng /unlock và /lock, cũng như cơ chế tự động hủy UID sau thời gian chờ đều đã được thực hiện thành công.

Bên cạnh đó, đề tài còn giúp củng cố và vận dụng hiệu quả các kiến thức đã học về vi điều khiển, hệ thống nhúng, truyền thông không dây, cảm biến, lập trình điều khiển và mô phỏng hệ thống. Đây là nền tảng quan trọng để tiếp tục phát triển các ứng dụng thông minh trong lĩnh vực IoT và tự động hóa trong tương lai.

Tuy nhiên, do giới hạn về thời gian, phạm vi thực hiện và điều kiện mô phỏng, đề tài hiện mới dừng ở mức mô hình thử nghiệm trên Wokwi. Một số chức năng nâng cao như giao diện web xác thực hoàn chỉnh, quản lý nhiều tài khoản người dùng, lưu lịch sử truy cập hoặc xác thực bằng Bluetooth vẫn chưa được triển khai đầy đủ. Dù vậy, kết quả đạt được cho thấy đề tài có tính thực tiễn và khả năng mở rộng cao.

5.2. Hướng phát triển

Từ những kết quả đã đạt được, hệ thống có thể tiếp tục được phát triển theo các hướng sau:

- Triển khai mô hình trên phần cứng thực tế để đánh giá độ ổn định trong môi trường sử dụng thật.
- Xây dựng giao diện web hoàn chỉnh để hỗ trợ nhập mã PIN hoặc quản lý người dùng trực tiếp trên hệ thống.
- Tích hợp thêm xác thực bằng Bluetooth để đáp ứng đầy đủ hơn yêu cầu mở rộng của đề tài.
- Thêm cảm biến trạng thái cửa để xác định chính xác cửa đang mở hay đóng trong thực tế.
- Xây dựng cơ chế quản lý nhiều người dùng với khả năng thêm, xóa hoặc cập nhật UID từ xa.
- Lưu lịch sử truy cập để phục vụ việc giám sát và kiểm tra an ninh.
- Tích hợp màn hình OLED hoặc LCD để hiển thị thông tin trạng thái trực tiếp trên thiết bị.
- Kết hợp thêm các phương thức xác thực hiện đại như vân tay hoặc mặt khẩu động để tăng tính bảo mật.
- Phát triển ứng dụng điện thoại hoặc nền tảng điều khiển riêng thay cho Telegram nhằm tăng tính chuyên nghiệp của hệ thống.

Nhìn chung, hệ thống khóa cửa thông minh dựa trên ESP32 và RFID là một hướng nghiên cứu có tính thực tiễn cao, phù hợp với xu hướng ứng dụng công nghệ IoT trong đời sống. Với việc tiếp tục cải tiến và mở rộng, mô hình này hoàn toàn có thể phát triển thành một giải pháp kiểm soát truy cập thông minh có khả năng ứng dụng trong thực tế.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, “Tài liệu Arduino dành cho ESP32”, truy cập tại: <https://docs.espressif.com/projects/arduino-esp32>

[2] Espressif Systems, “Hướng dẫn bắt đầu với Arduino ESP32”, truy cập tại: https://docs.espressif.com/projects/arduino-esp32/en/latest/getting_started.html

[3] Espressif Systems, “Tài liệu giao tiếp Wi-Fi trên ESP32”, truy cập tại: <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html>

[4] Espressif Systems, “Giới thiệu về ESP32 trong bộ tài liệu ESP-IDF”, truy cập tại: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/about.html>

[5] Miguel Balboa, “Thư viện RFID MFRC522 cho Arduino”, GitHub, truy cập tại: <https://github.com/miguelbalboa/rfid>

[6] Madhephaestus, “Thư viện ESP32Servo dùng để điều khiển servo trên ESP32”, GitHub tại: <https://github.com/madhephaestus/ESP32Servo>

[7] Witnessmenow, “Thư viện Universal Arduino Telegram Bot”, GitHub, truy cập tại: <https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot>

[8] Wokwi, “Tài liệu hướng dẫn sử dụng Wokwi”, truy cập tại: <https://docs.wokwi.com/>

[9] Wokwi, “Hướng dẫn mô phỏng ESP32 trên Wokwi”, truy cập tại: <https://docs.wokwi.com/guides/esp32>

[10] Wokwi, “Nền tảng mô phỏng vi điều khiển Wokwi”, truy cập tại: <https://wokwi.com/>

[10] Trường Đại học Khoa học - Đại học Huế. *Tài liệu bài giảng môn học Phát triển ứng dụng IoT.*

Link Github: https://github.com/vvdung-husc/2025-2026.2.TIN4024.002/tree/main/TEAM_09/NguyenGiaHuy_22t1020151/HT_KHOACUA_THONHMINH_ESP32_RFID

PHỤ LỤC:

```
#include <Arduino.h>
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>
#include <WiFi.h>
#include <WebServer.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>
```

```
// =====
```

```
// WIFI CONFIG
```

```
// =====
```

```
#define USE_WOKWI_WIFI true
```

```
const char* WIFI_SSID = USE_WOKWI_WIFI ? "Wokwi-
GUEST" : "Toi la Nobita";
```

```
const char* WIFI_PASSWORD = USE_WOKWI_WIFI ? "" :
"MatKhuWiFi";
```

```
const int WIFI_CHANNEL = 6;
```

```
// =====
```

```
// TELEGRAM CONFIG
```

```
// =====
```

```
const char* BOT_TOKEN = "7572820121:AAGaUNuQz_-
QeOI4SJqdptBvlZlQVOOdykk";
```

```
const char* CHAT_ID = "-1003990872264";
```

```
// =====
```

```
// HARDWARE PINS
```

```
// =====
```

```
#define SS_PIN 5
```

```
#define RST_PIN 22
```

```
#define SERVO_PIN 13
```

```

#define LED_GREEN 2
#define LED_RED 4
#define BUZZER_PIN 27

// =====
// LOCK CONFIG
// =====
const int LOCK_ANGLE = 0;
const int UNLOCK_ANGLE = 90;
const unsigned long DOOR_OPEN_TIME_MS = 5000;
const unsigned long AUTH_TIMEOUT_MS = 30000;
const unsigned long RFID_COOLDOWN_MS = 1200;
const unsigned long TELEGRAM_POLL_MS = 2000;

const String ACCESS_PIN = "123456";

// =====
// AUTHORIZED RFID UID
// =====
String allowedCards[] = {
    "01 02 03 04",
    "11 22 33 44",
    "55 66 77 88"
};

const int allowedCount = sizeof(allowedCards) /
sizeof(allowedCards[0]);

// =====
// OBJECTS
// =====
MFRC522 rfid(SS_PIN, RST_PIN);
Servo doorServo;
WebServer server(80);
WiFiClientSecure secureClient;

```


UniversalTelegramBot bot(BOT_TOKEN, secureClient);

// =====

// GLOBAL STATE

// =====

bool isDoorOpen = false;

unsigned long doorOpenedAt = 0;

bool pendingWifiAuth = false;

unsigned long authStartedAt = 0;

String currentUid = "";

unsigned long lastCardReadAt = 0;

unsigned long lastTelegramPoll = 0;

// =====

// HELPER

// =====

String getUIDString(MFRC522::Uid* uid) {

String result = "";

for (byte i = 0; i < uid->size; i++) {

if (uid->uidByte[i] < 0x10) result += "0";

result += String(uid->uidByte[i], HEX);

if (i < uid->size - 1) result += " ";

}

result.toUpperCase();

return result;

}

bool isAuthorizedCard(const String& uid) {

for (int i = 0; i < allowedCount; i++) {

if (uid == allowedCards[i]) return true;

}

return false;

}

```
bool isAuthSessionValid() {  
    return pendingWifiAuth && (millis() - authStartedAt <=  
AUTH_TIMEOUT_MS);  
}
```

```
void clearPendingAuth(bool turnOffRedLed = true) {  
    pendingWifiAuth = false;  
    currentUid = "";  
    if (turnOffRedLed) {  
        digitalWrite(LED_RED, LOW);  
    }  
}
```

```
String getSystemStatus() {  
    String s;  
    s += "Trang thai cua: ";  
    s += (isDoorOpen ? "DANG MO" : "DANG DONG");  
    s += "\nWiFi: ";  
    s += (WiFi.status() == WL_CONNECTED ? "DA KET NOI"  
: "MAT KET NOI");  
    s += "\nIP: ";  
    s += WiFi.localIP().toString();  
    s += "\nCho xac thuc WiFi: ";  
    s += (isAuthSessionValid() ? "CO" : "KHONG");  
    if (isAuthSessionValid()) {  
        s += "\nUID dang cho: ";  
        s += currentUid;  
    }  
    return s;  
}
```

```
void beepOK() {  
    tone(BUZZER_PIN, 2000, 150);  
}
```

```
void beepWait() {  
    tone(BUZZER_PIN, 1600, 400);  
}
```

```
void beepError() {  
    tone(BUZZER_PIN, 1000, 200);  
    delay(250);  
    tone(BUZZER_PIN, 800, 300);  
    delay(350);  
}
```

```
void sendTelegramMessage(const String& msg) {  
    if (WiFi.status() != WL_CONNECTED) return;  
    bot.sendMessage(CHAT_ID, msg, "");  
}
```

```
void lockDoor() {  
    doorServo.write(LOCK_ANGLE);  
    digitalWrite(LED_GREEN, LOW);  
    digitalWrite(LED_RED, LOW);  
    noTone(BUZZER_PIN);  
    isDoorOpen = false;  
    Serial.println("Door locked");  
}
```

```
void unlockDoor(const String& reason) {  
    doorServo.write(UNLOCK_ANGLE);  
    digitalWrite(LED_GREEN, HIGH);  
    digitalWrite(LED_RED, LOW);  
    beepOK();
```

```
    isDoorOpen = true;  
    doorOpenedAt = millis();
```

```
    clearPendingAuth(false);
```

```
Serial.println("Access granted - Door unlocked");  
sendTelegramMessage("Mo khoa thanh cong\nLy do: " +  
reason);  
}
```

```
void denyAccess(const String& reason) {  
    digitalWrite(LED_GREEN, LOW);  
    digitalWrite(LED_RED, HIGH);  
    beepError();  
}
```

```
Serial.println("Access denied: " + reason);  
sendTelegramMessage("Truy cap bi tu choi\nLy do: " +  
reason);  
}
```

```
void startWifiAuth(const String& uid) {  
    pendingWifiAuth = true;  
    authStartedAt = millis();  
    currentUid = uid;  
}
```

```
digitalWrite(LED_GREEN, LOW);  
digitalWrite(LED_RED, HIGH);  
beepWait();
```

```
String ip = WiFi.localIP().toString();
```

```
Serial.println("RFID hop le. Dang cho xac thuc WiFi...");  
Serial.print("UID dang cho: ");  
Serial.println(uid);  
Serial.print("Mo trinh duyet den: http://");  
Serial.println(ip);
```

```
String msg = "The RFID hop le: " + uid +  
    "\nDang cho xac thuc WiFi/PIN trong 30 giay." +
```

```

        "\nIP ESP32: http://" + ip;
    sendTelegramMessage(msg);
}

void connectWiFi() {
    Serial.print("Dang ket noi WiFi");
    WiFi.mode(WIFI_STA);

    if (USE_WOKWI_WIFI) {
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD,
WIFI_CHANNEL);
    } else {
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    }

    while (WiFi.status() != WL_CONNECTED) {
        delay(300);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi connected");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}

String htmlPage() {
    String html;
    html += "<!DOCTYPE html><html><head><meta
charset='UTF-8'>";
    html += "<meta name='viewport' content='width=device-
width, initial-scale=1.0'>";
    html += "<title>ESP32 Smart Lock</title>";
    html += "<style>";

```

```

    html += "body{font-family:Arial,sans-
    serif;background:#f5f7fb;margin:0;padding:24px;color:#222;}
    ";
    html += ".box{max-
    width:520px;margin:auto;background:#fff;padding:24px;bord
    er-radius:16px;box-shadow:0 10px 30px rgba(0,0,0,.08);}";
    html += "h1{margin-top:0;font-size:24px;}";
    html += ".ok{color:#0a8f3d;font-weight:bold;}";
    html += ".warn{color:#b86a00;font-weight:bold;}";
    html += ".bad{color:#c62828;font-weight:bold;}";
    html += "input{width:100%;padding:12px;font-
    size:16px;border:1px solid #ccc;border-radius:10px;box-
    sizing:border-box;}";
    html += "button{margin-
    top:12px;width:100%;padding:12px;background:#0b5ed7,col
    or:#fff;border:none;border-radius:10px;font-
    size:16px;cursor:pointer;}";
    html += "code{background:#f1f3f5;padding:2px 6px;border-
    radius:6px;}";
    html += "</style></head><body><div class='box'>";
    html += "<h1>ESP32 Smart Door Lock</h1>";
    html += "<p>WiFi: <span class='ok'>";
    html += (WiFi.status() == WL_CONNECTED ? "Da ket noi"
    : "Mat ket noi");
    html += "</span></p>";
    html += "<p>IP: <code>";
    html += WiFi.localIP().toString();
    html += "</code></p>";
    html += "<p>Cua hien tai: <strong>";
    html += (isDoorOpen ? "Dang mo" : "Dang dong");
    html += "</strong></p>";

    if (isAuthSessionValid()) {
        unsigned long remain = (AUTH_TIMEOUT_MS - (millis() -
        authStartedAt)) / 1000;

```

```

    html += "<p class='warn'>Da quet the hop le. Hay nhap PIN
trong ";
    html += String(remain);
    html += " giay.</p>";
    html += "<p>UID: <code>" + currentUid + "</code></p>";
    html += "<form action='/verify' method='POST'>";
    html += "<input type='password' name='pin'
placeholder='Nhap ma PIN'>";
    html += "<button type='submit'>Xac thuc mo
khoa</button>";
    html += "</form>";
} else {
    html += "<p class='bad'>Chua co the hop le nao dang cho
xac thuc.</p>";
    html += "<p>Hay quet the RFID truoc.</p>";
}

    html += "<hr>";
    html += "<p>Telegram commands: <code>/status</code>,
<code>/unlock</code>, <code>/lock</code>,
<code>/approve</code></p>";
    html += "</div></body></html>";
    return html;
}

// =====
// WEB HANDLERS
// =====
void handleRoot() {
    server.send(200, "text/html", htmlPage());
}

void handleVerify() {
    if (!pendingWifiAuth) {

```

```
server.send(403, "text/html", "<h2>Khong co phien xac thuc  
nao dang cho.</h2><a href='/'>Quay lai</a>");  
return;  
}
```

```
if (!isAuthSessionValid()) {  
    clearPendingAuth();  
    noTone(BUZZER_PIN);  
    server.send(408, "text/html", "<h2>Het thoi gian xac  
thuc.</h2><a href='/'>Quay lai</a>");  
    return;  
}
```

```
String pin = server.arg("pin");  
if (pin == ACCESS_PIN) {  
    String uid = currentUid;  
    unlockDoor("RFID + PIN web hop le\nUID: " + uid);  
    server.send(200, "text/html", "<h2>Mo khoa thanh  
cong.</h2><a href='/'>Quay lai</a>");  
} else {  
    String uid = currentUid;  
    denyAccess("Sai ma PIN web cho UID: " + uid);  
    clearPendingAuth();  
    noTone(BUZZER_PIN);  
    server.send(401, "text/html", "<h2>Sai PIN. Tu choi truy  
cap.</h2><a href='/'>Quay lai</a>");  
}  
}
```

```
void setupWebServer() {  
    server.on("/", HTTP_GET, handleRoot);  
    server.on("/verify", HTTP_POST, handleVerify);  
    server.begin();  
}
```

```
Serial.println("Web server started");
```



```

Serial.print("Truy cap: http://");
Serial.println(WiFi.localIP());
}

// =====
// TELEGRAM
// =====

void handleTelegramMessages(int count) {
  for (int i = 0; i < count; i++) {
    String chatId = bot.messages[i].chat_id;
    String text = bot.messages[i].text;
    String fromName = bot.messages[i].from_name;

    if (chatId != CHAT_ID) {
      bot.sendMessage(chatId, "Unauthorized user", "");
      continue;
    }

    Serial.print("Telegram message from ");
    Serial.print(fromName);
    Serial.print(": ");
    Serial.println(text);

    if (text == "/start") {
      String welcome = "ESP32 Smart Lock\n";
      welcome += "/status - xem trang thai\n";
      welcome += "/unlock - mo cua tu xa\n";
      welcome += "/lock - dong cua\n";
      welcome += "/approve - phe duyet neu dang cho xac thuc";
      bot.sendMessage(CHAT_ID, welcome, "");
    } else if (text == "/status") {
      bot.sendMessage(CHAT_ID, getSystemStatus(), "");
    } else if (text == "/lock") {
      lockDoor();
      bot.sendMessage(CHAT_ID, "Da dong cua.", "");
    }
  }
}

```

```

    } else if (text == "/unlock") {
        unlockDoor("Lenh Telegram /unlock");
        bot.sendMessage(CHAT_ID, "Da mo cua bang Telegram.",
            "");
    } else if (text == "/approve") {
        if (isAuthSessionValid()) {
            String uid = currentUid;
            unlockDoor("RFID + Telegram /approve\nUID: " + uid);
            bot.sendMessage(CHAT_ID, "Da phe duyet va mo cua.",
                "");
        } else {
            bot.sendMessage(CHAT_ID, "Khong co phien nao dang
cho xac thuc.", "");
        }
    } else {
        bot.sendMessage(CHAT_ID, "Lenh khong hop le. Dung
/start de xem menu.", "");
    }
}
}
}

```

```

void pollTelegram() {
    if (millis() - lastTelegramPoll < TELEGRAM_POLL_MS)
return;
    lastTelegramPoll = millis();

```

```

    int numNewMessages =
bot.getUpdates(bot.last_message_received + 1);
    while (numNewMessages) {
        handleTelegramMessages(numNewMessages);
        numNewMessages =
bot.getUpdates(bot.last_message_received + 1);
    }
}

```

```

// =====
// RFID
// =====

void handleRFID() {
    if (millis() - lastCardReadAt < RFID_COOLDOWN_MS)
return;
    if (pendingWifiAuth) return;
    if (!rfid.PICC_IsNewCardPresent()) return;
    Serial.println("Card present detected");
    if (!rfid.PICC_ReadCardSerial()) {
        Serial.println("Card present but UID read failed");
        return;
    }
    lastCardReadAt = millis();

    String uid = getUIDString(&rfid.uid);
    Serial.print("Card detected. UID: ");
    Serial.println(uid);

    if (isAuthorizedCard(uid)) {
        startWifiAuth(uid);
    } else {
        denyAccess("The RFID không hợp lệ: " + uid);
    }
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();
}

// =====
// SETUP
// =====

void setup() {
    Serial.begin(115200);

    pinMode(LED_GREEN, OUTPUT);

```

```
pinMode(LED_RED, OUTPUT);  
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(LED_GREEN, LOW);  
digitalWrite(LED_RED, LOW);  
noTone(BUZZER_PIN);
```

```
doorServo.attach(SERVO_PIN);  
lockDoor();
```

```
SPI.begin(18, 19, 23, 5);  
rfid.PCD_Init();
```

```
connectWiFi();  
secureClient.setInsecure();  
setupWebServer();
```

```
sendTelegramMessage("ESP32 Smart Lock da khoi  
dong.\nIP: http://" + WiFi.localIP().toString());
```

```
Serial.println("=====  
==");  
Serial.println("ESP32 SMART DOOR LOCK READY");  
Serial.println("1. Quet the RFID");  
Serial.println("2. Dung Telegram /approve de xac thuc");  
Serial.println("=====  
==");  
}
```

```
// =====  
// LOOP  
// =====
```

```
void loop() {  
    server.handleClient();  
    handleRFID();
```

```
pollTelegram();

if (pendingWifiAuth && !isAuthSessionValid()) {
    String expiredUid = currentUid;
    clearPendingAuth();
    noTone(BUZZER_PIN);
    sendTelegramMessage("Het han xac thuc cho UID: " +
expiredUid);
    Serial.println("Authentication timeout");
}

if (isDoorOpen && millis() - doorOpenedAt >=
DOOR_OPEN_TIME_MS) {
    lockDoor();
    sendTelegramMessage("Cua da tu dong dong lai.");
}

if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi mat ket noi. Dang thu ket noi lai...");
    connectWifi();
    setupWebServer();
}

delay(10);
}
```